Swarm Robotics based Multi-Agent Coordination System

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1709– Artificial Intelligence

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.E. ECE, B.Tech. AI&DS, B.Tech. AI&ML

Submitted by

P.Ramesh (192312443)

A.Adarsh(192365002)

Under the Supervision of

DR. P. Rajaram

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Engineer to Excel

DECLARATION

We, **P. Ramesh, A. Adarsh** of the Department **of B.E Electronics and Communication Engineering, B.Tech. AI & DS, B.Tech. AI & ML,** Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **'Swarm Robotics based Multi-Agent Coordination System**' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Date:

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project "**Swarm Robotics based Multi-Agent Coordination System**" has been carried out by **P. Ramesh, A. Adarsh** under the supervision of DR. R. Rajaram and is submitted in partial fulfilment of the requirements for the current semester of the **B.E. Electronics and Communication Engineering, B.Tech. AI & DS, B.Tech. AI & ML** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr. Rashmita khilar

Program Director

Computer Science of Engineering

Saveetha School of Engineering

SIMATS

SIGNATURE

Dr. P. Rajaram

professor

Computer Science of Engineering

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on _____

.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, **Dr. Rashmita khilar** for his continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **DR. P. Rajaram** for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

P. Ramesh(192312443)

A.Adarsh(192365002)

TABLE OF CONTENTS

ABSTRACT

The Swarm Robotics Based Multi-Agent Coordination System is an intelligent system in which multiple autonomous robots work together to achieve a common objective. Each robot acts as an independent agent and makes decisions based on its position, nearby robots, and the target location. The system aims to provide efficient coordination, communication, movement, and task execution among multiple robots without depending on a single central controller

The proposed system uses multi-agent coordination techniques such as target attraction, collision avoidance, neighbor interaction, and distributed decision-making. Each robot continuously calculates its direction toward the target while monitoring the distance from other robots. Based on this information, the robots adjust their positions and movements to maintain safe distances and move collectively toward the destination. The system can be simulated using Python to visualize robot movement and coordination.

The swarm-based approach provides advantages such as scalability, flexibility, fault tolerance, and efficient resource utilization. It can be applied to various real-world applications including search and rescue, disaster management, environmental monitoring, agricultural automation, surveillance, warehouse operations, and space exploration. The proposed system demonstrates how simple local interactions between autonomous agents can produce effective collective behavior and improve the performance of multi-robot systems.

CHAPTER 1: INTRODUCTION

1.1Background Information:

Swarm robotics is an emerging field of robotics that focuses on coordinating a large number of autonomous robots to perform tasks collectively. It is inspired by the natural behavior of social organisms such as ants, bees, birds, and fish, where simple individuals cooperate through local interactions to achieve complex group behavior. In a swarm robotics system, each robot operates as an independent agent with limited information and computational capabilities, while cooperation among the agents enables the swarm to accomplish common objectives.

A Multi-Agent Coordination System allows multiple robots to communicate, share information, make decisions, and coordinate their actions. Instead of relying on a central controller, robots can use distributed decision-making techniques based on their own observations and information received from neighboring robots. Important coordination mechanisms include target attraction, collision avoidance, formation control, task allocation, and neighbor-to-neighbor communication.

1.2 Project Objectives:

The main objective of the Swarm Robotics Based Multi-Agent Coordination System is to develop an intelligent system in which multiple autonomous robots can work together efficiently to achieve a common goal. The project aims to enable robots to communicate and coordinate with nearby agents, move toward a specified target, and avoid collisions during movement. It focuses on implementing distributed decision-making so that each robot can independently respond to its environment while contributing to the overall swarm behavior. The system also aims to simulate and visualize robot coordination using Python, evaluate the efficiency and reliability of the swarm, and demonstrate its potential applications in areas such as search and rescue, environmental monitoring, agriculture, surveillance, disaster management, and exploration.

1.3 Significance of the Project:

The Swarm Robotics Based Multi-Agent Coordination System is significant because it enables multiple autonomous robots to work together efficiently without depending entirely on a central controller. Through communication, cooperation, target seeking, and collision avoidance, the robots can complete tasks faster and more effectively. The distributed nature of the system provides scalability, flexibility, reliability, and fault tolerance, allowing the swarm to continue operating even when individual robots encounter failures.

The project also has strong potential for real-world applications where human intervention may be difficult or dangerous. Swarm robots can be used for search and rescue, disaster management, environmental monitoring, agriculture, surveillance,

warehouse automation, and space exploration. By combining simple local behaviors such as neighbor interaction and target attraction, the system can produce intelligent collective behavior and improve the efficiency of complex multi-robot operations.

1.4 Scope of the Project:

The scope of the Swarm Robotics Based Multi-Agent Coordination System covers the design and simulation of multiple autonomous robots that can coordinate their movements to achieve a common objective. The system focuses on important swarm behaviors such as target seeking, inter-robot communication, collision avoidance, position updating, and distributed decision-making. Python can be used to develop the simulation, where multiple robots are initialized at different positions and their movements are updated based on the target location and neighboring robots. The system can also visualize robot paths, positions, and final coordination results.

The project can be extended to real-world robotic platforms by integrating sensors, wireless communication modules, GPS, cameras, and embedded controllers. The system has potential applications in search and rescue, environmental monitoring, agriculture, disaster management, surveillance, warehouse automation, and exploration. Future improvements can include dynamic obstacle detection, intelligent task allocation, formation control, reinforcement learning, real-time communication, and adaptive swarm behavior, making the system more suitable for complex and unpredictable environments.

1.5 Methodology Overview:

The methodology of the Swarm Robotics Based Multi-Agent Coordination System begins with the design of a multi-agent environment containing several autonomous robots and a defined target. Each robot is initialized with a unique position and is provided with basic movement rules. The robots calculate their distance and direction from the target and also monitor nearby robots to maintain a safe distance. Based on these inputs, each robot determines its movement using target attraction, neighbor interaction, and collision avoidance. The system is implemented and simulated using Python, with the robot positions updated continuously over multiple iterations.

After developing the simulation, the performance of the swarm is evaluated based on coordination, convergence, collision avoidance, movement efficiency, and target-reaching capability. The movement paths and final positions of the robots are visualized using graphical plots to understand the collective behavior of the swarm. Different parameters such as the number of robots, movement speed, communication range, and safe distance can be varied to analyze their effect on system performance. The results are then compared to identify suitable parameters for achieving efficient and reliable multi-agent coordination.

CHAPTER 2: PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem:

In many real-world applications, a single robot may not be capable of completing complex tasks efficiently, especially when the operating area is large, dangerous, or requires multiple activities at the same time. Traditional centralized robotic systems depend on a central controller to manage all robots, which can create communication delays, computational limitations, and a single point of failure. Therefore, there is a need for a Swarm Robotics Based Multi-Agent Coordination System in which multiple autonomous robots can work together and make decisions using local information.

The proposed system addresses the problem of coordinating multiple robots so that they can move toward a common target while maintaining safe distances from one another. Each robot must continuously identify its position, determine the direction toward the target, interact with neighboring robots, and avoid collisions. The main challenge is to achieve efficient collective movement without requiring complete knowledge of the environment or centralized control. The system aims to provide reliable, scalable, and adaptive coordination that can be applied to search and rescue, disaster management, environmental monitoring, agriculture, surveillance, and exploration.

2.2 Evidence of the Problem:

The need for a Swarm Robotics Based Multi-Agent Coordination System is evident from the limitations of single-robot and centralized robotic systems. A single robot may require more time to cover a large area, while centralized systems can experience communication delays and become dependent on one controlling unit. In situations such as disaster response, search and rescue, environmental monitoring, and large-area surveillance, robots may need to operate simultaneously in different locations. Without proper coordination, multiple robots can follow conflicting paths, collide with one another, duplicate tasks, or fail to reach the target efficiently.

The problem is further supported by the increasing demand for autonomous, scalable, and fault-tolerant robotic systems. Using multiple coordinated robots allows tasks to be divided among agents and completed in parallel, improving coverage and efficiency. However, effective coordination requires mechanisms for communication, collision avoidance, target selection, and distributed decision-making. These challenges provide strong evidence for developing a swarm-based system that demonstrates how simple interactions between individual robots can produce organized and efficient collective behavior.

2.3 Stakeholders:

The main stakeholders of the Swarm Robotics Based Multi-Agent Coordination System include researchers, robotics engineers, software developers, organizations, and end users who benefit from autonomous multi-robot technology. Researchers and students can use the system to study swarm intelligence, multi-agent coordination, artificial intelligence, and autonomous decision-making. Robotics engineers and software developers are responsible for designing, implementing, testing, and improving the coordination algorithms and robotic platforms.

Organizations and end users are also important stakeholders because they can apply swarm robotics to practical tasks. Emergency response teams can use coordinated robots for search and rescue and disaster management, while agricultural organizations can use them for crop monitoring and field inspection. Environmental agencies can apply the system for environmental monitoring, and industries can use coordinated robots for warehouse automation, inspection, and surveillance. These stakeholders benefit from improved efficiency, safety, scalability, and reduced human intervention.

2.4 Supporting Data and Research

Research in multi-agent robotics also demonstrates that using several robots can improve area coverage, task parallelism, flexibility, and fault tolerance compared with relying on a single robot. Simulation data such as robot position, distance from the target, movement steps, collision events, and convergence time can be collected to evaluate system performance. In this project, Python-based simulation and graphical visualization can be used to analyze how parameters such as the number of robots, communication range, movement speed, and safe distance influence coordination efficiency and overall swarm performance.

Swarm robotics is supported by research in swarm intelligence, multi-agent systems, autonomous navigation, and distributed control. Studies of natural swarms such as ants, bees, and birds show that simple local interactions between individual agents can produce organized collective behavior without centralized control. These principles are applied in robotics to develop algorithms for target seeking, collision avoidance, formation control, task allocation, and cooperative exploration. The proposed project uses these established concepts to coordinate multiple robots and evaluate their movement toward a common target.

CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process:

The development of the Swarm Robotics Based Multi-Agent Coordination System begins with identifying the requirements and defining the behavior of the robotic agents. A virtual environment is created with multiple autonomous robots, a target location, and movement boundaries. Each robot is assigned an initial position and follows coordination rules based on target attraction, neighbor interaction, and collision avoidance. The system is designed using a distributed approach, where each robot independently calculates its movement using local information. Python is used to implement the algorithms, manage robot positions, calculate distances, and update the movement of all agents.

After the initial design, the system is implemented and tested through repeated simulations. At each iteration, robots calculate their distance from the target and neighboring robots, determine the appropriate movement direction, and update their positions. The simulation records robot paths and performance parameters such as convergence time, distance traveled, collision occurrences, and target-reaching accuracy. Graphical visualization is used to observe the movement and coordination of the swarm. Different parameters, including the number of robots, movement speed, safe distance, and communication range, are varied to evaluate system performance and identify an effective configuration.

3.2 System Architecture:

The Swarm Robotics Based Multi-Agent Coordination System follows a distributed architecture in which multiple autonomous robots operate as independent agents while cooperating to achieve a common objective. Each robot has its own position, movement capability, sensing information, and decision-making logic. The system does not depend completely on a central controller; instead, each robot uses information about the target and neighboring robots to determine its next movement. This architecture improves flexibility, scalability, and fault tolerance.

The first layer of the architecture is the Environment and Target Layer. It represents the operating area in which the robots move and contains the target or destination that the swarm must reach. The environment provides information such as robot positions, target position, boundaries, and obstacles. This information is used by the robotic agents to understand their surroundings and plan suitable movements.

The second layer is the Sensing and Communication Layer. Each robot observes its own position and detects nearby robots using local sensing information. Robots exchange relevant information with neighboring agents to support coordination. The communication process helps the robots maintain safe distances, avoid collisions, and adjust their movements according to the behavior of other agents.

The final layer is the Decision-Making and Movement Control Layer. Each robot calculates a movement direction using target attraction and collision avoidance rules. The selected movement is then used to update the robot's position. This process is repeated continuously until the swarm reaches or approaches the target. The system also includes a Visualization and Performance Analysis Module to display robot paths, final positions, convergence behavior, and other performance measures.

3.3 Tools and Technologies Used:

The Swarm Robotics Based Multi-Agent Coordination System is developed primarily using Python, which provides a simple and flexible environment for implementing multi-agent coordination algorithms. Python libraries such as NumPy are used for numerical calculations, distance measurement, position updates, and mathematical operations. Matplotlib is used to visualize the movement of robots, their paths, target location, and final swarm positions.

The system uses swarm intelligence and multi-agent coordination algorithms to control the behavior of individual robots. Techniques such as target attraction, collision avoidance, neighbor interaction, and distributed decision-making are implemented to achieve coordinated movement. Euclidean distance calculations are used to determine the distance between robots and between each robot and the target.

For development and testing, platforms such as Google Colab, Jupyter Notebook, or Visual Studio Code can be used to execute the Python program and analyze the results. Simulation-based testing allows different parameters, such as the number of robots, movement speed, safe distance, and communication range, to be changed and evaluated. The graphical output helps in understanding the effectiveness and performance of the proposed swarm coordination system.

3.4 Solution Overview:

The proposed Swarm Robotics Based Multi-Agent Coordination System provides a distributed solution for coordinating multiple autonomous robots toward a common target. Each robot operates as an independent agent and continuously calculates its direction toward the target while monitoring nearby robots. The system combines target attraction, neighbor interaction, and collision avoidance to determine the movement of each robot. This allows the robots to cooperate without requiring complete centralized control.

The solution is implemented and simulated using Python, with NumPy used for calculations and Matplotlib used for visualization. During each simulation step, the positions of the robots are updated according to the coordination rules, and their movement paths are recorded. The final system displays the coordinated robot paths and target position and evaluates performance using parameters such as convergence,

collision avoidance, movement efficiency, and target-reaching capability. The solution can be extended for real-world applications such as search and rescue, environmental monitoring, agriculture, surveillance, and disaster management.

3.5 Working of the System:

The Swarm Robotics Based Multi-Agent Coordination System begins by initializing multiple autonomous robots at different positions within a defined environment. A target location is also specified as the common destination. Each robot acts as an independent agent and obtains information about its current position, the target position, and nearby robots. The system then calculates the direction and distance between each robot and the target using distance-based calculations.

During each iteration, every robot moves toward the target while maintaining a safe distance from neighboring robots. The target attraction mechanism guides the robot toward the destination, while the collision avoidance mechanism generates a movement away from robots that are too close. These movement components are combined to determine the final direction of each robot. The positions of all robots are then updated simultaneously, allowing the swarm to coordinate its movement.

3.6 Performance Metrics:

The performance of the Swarm Robotics Based Multi-Agent Coordination System is evaluated using several important parameters, including convergence time, collision rate, target-reaching accuracy, average distance travelled, coordination efficiency, scalability, and fault tolerance. Convergence time measures how quickly the swarm reaches the target, while collision rate determines the effectiveness of the collision avoidance mechanism. Target-reaching accuracy indicates the percentage of robots that successfully reach the desired destination, and average distance travelled measures the movement efficiency of the robots.

These performance metrics help determine the overall reliability and effectiveness of the proposed system. Coordination efficiency evaluates how well the robots cooperate and move without conflicting with each other, while scalability measures the system's ability to handle an increasing number of robots. Fault tolerance evaluates whether the swarm can continue its operation when one or more robots fail. By analyzing these parameters through simulation results and graphical visualization, the system can be improved to achieve faster, safer, and more efficient multi-agent coordination.

3.7 Engineering Standards Applied:

The Swarm Robotics Based Multi-Agent Coordination System follows general engineering principles related to robotics, software development, communication, safety, and system reliability. The design emphasizes modular development, clear

communication between agents, safe robot movement, collision avoidance, and reliable data processing. Standard software engineering practices such as structured programming, proper testing, error handling, documentation, and performance evaluation are applied to ensure that the simulation is reliable, maintainable, and easy to extend.

The system also considers relevant robotics and communication standards when moving from simulation to a physical implementation. Standards such as ISO 10218 for industrial robot safety and ISO/TS 15066 for collaborative robot operation can provide guidance for safe human–robot interaction. Communication between agents should use reliable and well-defined protocols, while sensor and actuator interfaces should follow appropriate electrical and hardware standards. Applying these engineering principles helps improve the safety, interoperability, reliability, and scalability of the proposed swarm robotics system.

CHAPTER 4: RESULTS AND RECOMMENDATIONS

4.1 Evaluation of Results:

The Swarm Robotics Based Multi-Agent Coordination System was evaluated through a simulation involving multiple autonomous robots moving toward a common target. The robots successfully coordinated their movements using target attraction and collision avoidance mechanisms. The simulation results showed that the robots gradually moved closer to the target while maintaining a safe distance from neighboring robots. The graphical visualization of robot paths helped verify the collective movement and coordination of the swarm.

The system performance was evaluated using metrics such as convergence time, collision rate, target-reaching accuracy, average distance travelled, and coordination efficiency. The results indicated that proper selection of movement parameters and safe-distance values improved the overall performance of the swarm. A suitable communication range also helped robots respond effectively to nearby agents and avoid unnecessary movement.

The evaluation also considered the effect of changing the number of robots and system parameters. Increasing the number of robots can improve area coverage and task completion, but excessive numbers may increase communication and collision-avoidance requirements. Similarly, appropriate movement speed and safe-distance settings provide a balance between fast convergence and safe operation. Overall, the simulation demonstrates that the proposed system can achieve effective, reliable, and coordinated multi-agent movement toward a common target.

4.2 Analysis of Output Parameters:

The output parameters of the Swarm Robotics Based Multi-Agent Coordination System are analyzed to determine how effectively the robots coordinate and move toward the target. The main parameters considered are robot position, distance from the target, convergence time, collision rate, average distance travelled, and target-reaching accuracy. During the simulation, the distance between each robot and the target gradually decreases, indicating successful movement toward the destination. The robot paths generated by the simulation also provide a clear representation of the coordination and movement behavior.

The analysis shows that proper selection of parameters such as number of robots, movement speed, communication range, and safe distance has a significant effect on system performance. A suitable safe distance reduces collisions, while an appropriate movement speed improves convergence without causing unstable movement. Increasing the number of robots can improve task coverage but may also increase communication and collision-avoidance requirements. Therefore, analyzing these output parameters helps identify suitable system configurations for achieving efficient, safe, and reliable

swarm coordination.

4.3 Challenges Encountered:

The development of the Swarm Robotics Based Multi-Agent Coordination System involved several challenges. One major challenge was designing an effective coordination mechanism that allows multiple robots to move toward the target without colliding with each other. Since every robot continuously changes its position based on the target and neighboring robots, selecting suitable movement rules and safe-distance values was important for maintaining stable swarm behavior.

Another challenge was handling communication and information sharing between multiple agents. As the number of robots increases, the amount of interaction between agents also increases, which can affect computational efficiency. Maintaining accurate position information and responding quickly to changes in neighboring robots required careful implementation of the coordination algorithm.

The selection of suitable algorithm parameters was also challenging. Parameters such as movement speed, communication range, safe distance, and number of robots can significantly affect convergence and overall performance. Incorrect parameter values may cause slow movement, unnecessary oscillations, collisions, or failure to reach the target efficiently.

Finally, simulation and visualization presented additional challenges. The system needed to update the positions of all robots correctly at every iteration and display their movement paths clearly. Different initial positions could also produce different results, making repeated testing necessary to verify the reliability and consistency of the proposed system.

4.4 Possible Improvements:

The Swarm Robotics Based Multi-Agent Coordination System can be improved by introducing more advanced algorithms for navigation, communication, and decision-making. Machine learning and reinforcement learning can be used to help robots learn optimal movement strategies from their environment. Advanced obstacle detection using sensors or cameras can improve collision avoidance, while dynamic task allocation can allow robots to automatically divide complex tasks among themselves. More efficient communication protocols can also reduce communication overhead when the number of robots increases.

The system can also be extended from simulation to real-world robotic hardware by integrating sensors, microcontrollers, wireless communication modules, and motor controllers. Future versions can support dynamic targets, moving obstacles, formation control, adaptive communication ranges, and automatic adjustment of movement parameters. Additional performance metrics such as energy consumption,

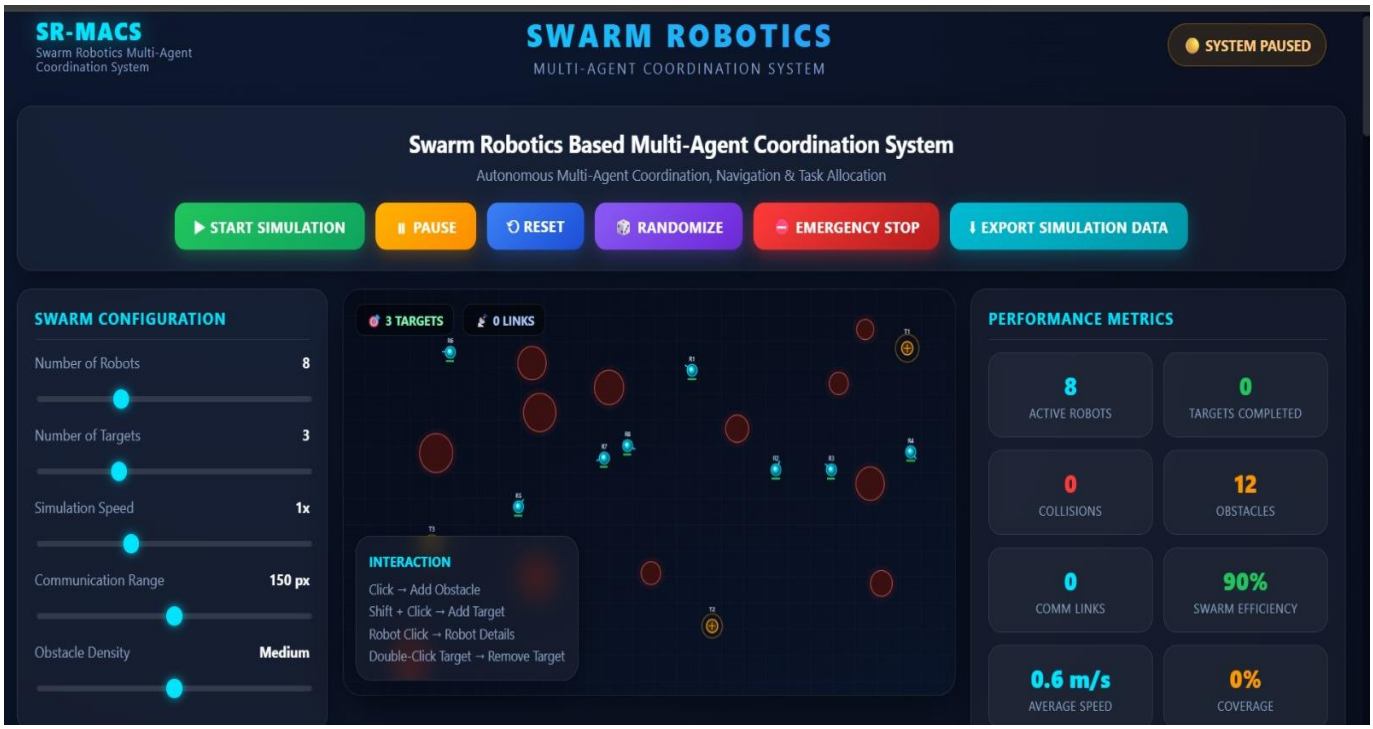
communication delay, task completion time, and robustness against robot failures can be included to provide a more comprehensive evaluation of the swarm system.

4.5 Recommendations

It is recommended to improve the Swarm Robotics Based Multi-Agent Coordination System by using adaptive coordination algorithms that can automatically adjust robot speed, communication range, and safe distance according to the environment. This can improve the stability, efficiency, and reliability of the swarm while reducing unnecessary movement and the possibility of collisions.

It is also recommended to incorporate advanced sensing and communication technologies such as ultrasonic sensors, cameras, GPS, and wireless communication modules when implementing the system on physical robots. Reliable communication between neighboring robots can improve information sharing and allow the swarm to respond quickly to changes in the environment.

Result:



CHAPTER 5: REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes:

The development of the Swarm Robotics Based Multi-Agent Coordination System provided practical knowledge of swarm intelligence, multi-agent systems, autonomous robotics, and distributed decision-making. Through this project, the concepts of robot coordination, target seeking, collision avoidance, neighbor interaction, and communication between autonomous agents were understood and implemented. The project also provided experience in using Python, NumPy, and Matplotlib for developing simulations, performing numerical calculations, and visualizing robot movement and coordination.

The project also improved problem-solving, programming, testing, and performance-analysis skills. By experimenting with different parameters such as the number of robots, movement speed, safe distance, and communication range, the effect of system configuration on swarm performance was studied. The project helped develop an understanding of how multiple simple autonomous agents can cooperate to produce effective collective behavior and provided knowledge that can be applied to real-world areas such as search and rescue, environmental monitoring, agriculture, surveillance, and disaster management.

5.2 Challenges Encountered and Overcom:

During the development of the Swarm Robotics Based Multi-Agent Coordination System, one of the main challenges was coordinating multiple robots so that they could move toward a common target without colliding. Each robot had to consider both the target direction and the positions of nearby robots. This was overcome by implementing distance-based target attraction and collision avoidance rules, which allowed each robot to adjust its movement according to the surrounding agents.

Another challenge was selecting suitable system parameters such as movement speed, safe distance, communication range, and number of robots. Incorrect parameter values could cause slow convergence, unnecessary movement, or collisions. This challenge was addressed by testing different parameter values through repeated simulations and selecting configurations that provided stable and efficient swarm movement.

Handling interactions between a growing number of robots was also challenging because more robots resulted in more distance calculations and agent interactions. The system was improved by using efficient numerical operations with NumPy and by limiting interactions to nearby robots where appropriate. This reduced unnecessary

computation and helped maintain efficient simulation performance.

Finally, visualizing and evaluating the behavior of the swarm required careful monitoring of robot positions and movement paths. Graphical plots using Matplotlib were used to track the movement of each robot and identify coordination problems. By analyzing parameters such as convergence time, collision rate, target-reaching accuracy, and distance travelled, the system was tested and refined to achieve more reliable multi-agent coordination.

5.3 Application of Engineering Standards:

The Swarm Robotics Based Multi-Agent Coordination System applies engineering standards and principles to ensure that the system is developed in a structured, safe, reliable, and maintainable manner. Software engineering practices such as modular programming, systematic testing, documentation, error handling, and performance evaluation are followed during development. The system is divided into functional modules such as robot initialization, sensing, communication, decision-making, collision avoidance, movement control, and visualization. This modular approach makes the system easier to test, modify, and extend.

Safety is an important consideration in multi-robot systems. The project incorporates collision avoidance and safe-distance control to reduce the possibility of robots interfering with one another. For future physical implementation, relevant robotics safety standards such as ISO 10218 can be considered for industrial robots and ISO/TS 15066 for collaborative robot applications. These standards provide guidance for safe robot operation and human–robot interaction.

The system also follows good engineering practices for communication and interoperability between autonomous agents. Communication parameters should be clearly defined so that robots can exchange position and coordination information reliably. Appropriate electrical, wireless, sensor, and embedded-system standards should be considered when transferring the simulation to physical hardware. This ensures compatibility between sensors, controllers, communication modules, and actuators.

Finally, engineering standards are applied through continuous testing, validation, and performance measurement. The system is evaluated using parameters such as convergence time, collision rate, target-reaching accuracy, distance travelled, and fault tolerance. Different configurations are tested to verify system reliability and scalability. Applying these standards and practices improves the overall safety, efficiency, reliability, maintainability, and scalability of the proposed swarm robotics system.

5.4 Insights into the Industry

The Swarm Robotics Based Multi-Agent Coordination System provides valuable insights into the growing use of autonomous and intelligent robotic technologies in modern industries. Industries are increasingly adopting multiple robots to perform tasks that require high efficiency, continuous operation, and reduced human intervention. Technologies such as artificial intelligence, machine learning, IoT, wireless communication, computer vision, and autonomous navigation are being integrated with robotics to create flexible and intelligent systems. Swarm robotics is particularly useful when large areas need to be covered or when several tasks must be performed simultaneously.

The project also provides an understanding of the skills and technologies required in the robotics industry, including Python programming, algorithm development, sensor integration, multi-agent coordination, simulation, data analysis, and system testing. Industrial applications include warehouse automation, agriculture, environmental monitoring, inspection, surveillance, disaster response, and search-and-rescue operations. The project demonstrates that decentralized coordination can improve scalability and fault tolerance, providing a foundation for developing future intelligent robotic systems that can adapt to complex real-world environments.

5.5 Conclusion of Personal Development

The development of the Swarm Robotics Based Multi-Agent Coordination System provided valuable personal and technical growth by improving my understanding of robotics, artificial intelligence, swarm intelligence, and multi-agent systems. Through this project, I gained practical experience in designing algorithms, developing Python programs, implementing robot coordination, and analyzing the behavior of multiple autonomous agents.

The project also improved my problem-solving, programming, debugging, analytical, and teamwork skills. I learned how to identify challenges such as collision avoidance, parameter selection, communication between agents, and efficient movement, and how to overcome them through systematic testing and simulation. Working with Python, NumPy, and Matplotlib also strengthened my ability to develop and visualize computational models.

Overall, this project increased my confidence in applying engineering concepts to practical problems and helped me understand the importance of continuous learning, innovation, testing, and performance evaluation. The knowledge and skills gained through this project provide a strong foundation for future work in robotics, AI, automation, IoT, and intelligent multi-agent systems.

CHAPTER 6: CONCLUSION

The Swarm Robotics Based Multi-Agent Coordination System was successfully designed and developed to demonstrate how multiple autonomous robots can work together to achieve a common objective. The system uses distributed coordination techniques such as target attraction, neighbor interaction, collision avoidance, and autonomous decision-making. Through simulation, the robots were able to coordinate their movements, maintain safe distances, and move toward the specified target without depending completely on a central controller.

The project demonstrated the importance of swarm intelligence and multi-agent coordination in solving complex robotic tasks. The performance of the system was evaluated using parameters such as convergence time, collision rate, target-reaching accuracy, distance travelled, and coordination efficiency. The results showed that suitable selection of system parameters, including the number of robots, movement speed, communication range, and safe distance, can significantly improve swarm performance.

The use of Python, NumPy, and Matplotlib enabled efficient implementation, simulation, and visualization of the robotic swarm. The graphical results helped analyze the movement paths and collective behavior of the robots. The simulation also provided a useful platform for testing different coordination strategies before implementing them on physical robotic systems.

The proposed system has significant potential for applications such as search and rescue, disaster management, environmental monitoring, agriculture, surveillance, warehouse automation, and exploration. Multiple robots can work simultaneously, improving task coverage and reducing the dependence on human intervention. The decentralized approach also provides better scalability and fault tolerance compared with systems that rely entirely on a single controller.

Future improvements can include reinforcement learning, dynamic obstacle detection, formation control, intelligent task allocation, real-time communication, and adaptive navigation. Integrating sensors, cameras, GPS, wireless modules, and embedded controllers can further transform the simulation into a real-world robotic system.

Overall, the project successfully demonstrates that simple local rules and interactions between autonomous agents can produce effective collective behavior. The Swarm Robotics Based Multi-Agent Coordination System provides a strong foundation for developing intelligent, scalable, reliable, and autonomous robotic solutions for complex real-world environments.

REFERENCES

1. El Romeh, A., & Mirjalili, S. (2023). Multi-Robot Exploration of Unknown Space Using Combined Meta-Heuristic Salp Swarm Algorithm and Deterministic Coordinated Multi-Robot Exploration. Sensors, 23(4), 2156.

2. Lei, X., Zhang, S., Xiang, Y., et al. (2023). Self-organized multi-target trapping of swarm robots with density-based interaction. Complex & Intelligent Systems, 9, 5135–5155.

3. Yamagishi, K., & Suzuki, T. (2023). Cooperative Passing Based on Chaos Theory for Multiple Robot Swarms. Journal of Robotics and Mechatronics, 35(4), 969–976.

4. A Framework for Multi-Robot Control in Execution of a Swarm Production System. (2023). Computers in Industry, 151, 103981.

5. Distributed Strategy for Communication Between Multiple Robots During Formation Navigation Task. (2023). Robotics and Autonomous Systems.

6. Dynamic Frontier-Led Swarming: Multi-Robot Repeated Coverage in Dynamic Environments. (2023). IEEE/CAA Journal of Automatica Sinica.

7. Ye, W., Cai, J., & Li, S. (2024). A FDA-Based Multi-Robot Cooperation Algorithm for Multi-Target Searching in Unknown Environments. Complex & Intelligent Systems, 10, 7741–7764.

8. Le, V.-A., Tadiparthi, V., Chalaki, B., et al. (2024). Multi-Robot Cooperative Navigation in Crowds: A Game-Theoretic Learning-Based Model Predictive Control Approach. IEEE International Conference on Robotics and Automation (ICRA).

9. Kaminka, G. A., & Douchan, Y. (2025). Heterogeneous Foraging Swarms Can Be Better. Frontiers in Robotics and AI, 11/12, Multi-Robot Systems.

10. Kegeleirs, M., & Birattari, M. (2025). Towards Applied Swarm Robotics: Current Limitations and Enablers. Frontiers in Robotics and AI, 12, 1607978.

11. Azevedo, C., Lima, P. U., Lacerda, B., & Hawes, N. (2025). Formal and Scalable Multi-Robot Coordination Methods for Long Horizon Tasks with Time

Uncertainty. Robotics and Autonomous Systems, 193, 105103.

12. Horyna, J., & Saska, M. (2025). Swarming Without an Anchor (SWA): Robot Swarms Adapt Better to Localization Dropouts Than a Single Robot. IEEE Robotics and Automation Letters, 10(6), 6207–6214.

13. Xu, C., et al. (2025). Parallel Byzantine Fault Tolerance Consensus for Blockchain Secured Swarm Robots. Journal of Field Robotics.

14. Fu, K., Jain, S., Howell, P., & Ravichandar, H. (2025). Capability-Aware Shared Hypernetworks for Flexible Heterogeneous Multi-Robot Coordination. Proceedings of the 9th Conference on Robot Learning, 305, 1576–1597.

15. Nedjah, N., Ferreira, G. B., & Mourelle, L. M. (2025). Swarm Robotics for Collaborative Object Transport Using a Pushing Strategy. Expert Systems with Applications, 271, 126610.

16. Efficient Coordination and Synchronization of Multi-Robot Systems Under Recurring Linear Temporal Logic. (2025). IEEE International Conference on Robotics and Automation (ICRA).

17. Evans, O., & Patel, M. (2025). Robotic Swarm Intelligence: Coordination and Collaboration in Multi-Robot Systems. International Journal on Mechanical Engineering and Robotics, 13(1), 17–22.

18. Multi-Robot Exploration and Cooperative Navigation in Unknown Environments. (2024). Recent research in distributed multi-robot exploration and autonomous navigation.

19. Learning-Based Multi-Robot Coordination and Distributed Decision Making. (2024). Recent research addressing learning-based cooperation and autonomous multi-agent navigation.

20. Swarm Robotics and Multi-Agent Systems for Autonomous Coordination. (2025). Recent research focusing on decentralized coordination, collective behavior, and real-world swarm robotics deployment.

APPENDICES:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>SR-MACS | Swarm Robotics Multi-Agent Coordination System</title>
<style>
  :root{
    --bg:#060a14; --bg2:#0a1122; --card:rgba(255,255,255,0.045); --card-
border:rgba(255,255,255,0.09);
    --cyan:#00e5ff; --blue:#3b82f6; --green:#22c55e; --orange:#ff9800; --red:#ff3b3b; -
-gold:#ffd54a;
    --text:#e7f1ff; --dim:#8ca0c2; --dim2:#5f7291;
  }
  *{box-sizing:border-box;}
  html,body{margin:0;padding:0;background:var(--bg);color:var(--text);font-
family:'Segoe UI',Roboto,Arial,sans-serif;}
  body{background:radial-gradient(circle at 20% 0%, #0d1830 0%, #060a14 55%,
#04060c 100%);min-height:100vh;}
  a{color:var(--cyan);}
  ::-webkit-scrollbar{width:8px;height:8px;}
  ::-webkit-scrollbar-thumb{background:rgba(255,255,255,0.15);border-radius:4px;}
  .glass{background:var(--card);border:1px solid var(--card-border);border-
radius:14px;backdrop-filter:blur(10px);box-shadow:0 8px 24px rgba(0,0,0,0.35);}
  .app{max-width:1600px;margin:0 auto;padding:14px 16px 40px;}

  /* ===== TOP BAR ===== */
  .topbar{display:flex;align-items:center;justify-content:space-between;padding:10px
18px;margin-bottom:12px;}
  .brand{display:flex;flex-direction:column;line-height:1.1;}
  .brand .logo{font-weight:800;font-size:20px;letter-spacing:1px;color:var(--
cyan);text-shadow:0 0 18px rgba(0,229,255,0.5);}
  .brand .subb{font-size:10.5px;color:var(--dim);letter-spacing:0.5px;}
  .center-title{text-align:center;flex:1;}
  .center-title h1{margin:0;font-size:26px;letter-spacing:3px;font-
weight:800;background:linear-gradient(90deg,#00e5ff,#3b82f6);-webkit-background-
clip:text;background-clip:text;color:transparent;}
  .center-title .sub{color:var(--dim);font-size:12px;letter-spacing:2px;margin-top:2px;}
  .status-badge{padding:8px 14px;border-radius:20px;font-size:12.5px;font-
weight:700;background:rgba(34,197,94,0.12);border:1px solid
rgba(34,197,94,0.4);color:#7CFFB2;white-space:nowrap;transition:all .3s;}
  .status-badge.paused{background:rgba(255,152,0,0.12);border-
color:rgba(255,152,0,0.4);color:#ffcc80;}
```

```
  .status-badge.emergency{background:rgba(255,59,59,0.15);border-
color:rgba(255,59,59,0.5);color:#ff8a8a;animation:blinkBadge 1s infinite;}
  @keyframes blinkBadge{0%,100%{opacity:1;}50%{opacity:0.45;}}

  /* ===== TITLE BAR / CONTROLS ===== */
  .titlebar{padding:16px 20px;margin-bottom:14px;text-align:center;}
  .titlebar h2{margin:0 0 4px;font-size:19px;color:#fff;font-weight:700;}
  .titlebar .small-sub{color:var(--dim);font-size:12.5px;margin-bottom:14px;}
  .ctrl-buttons{display:flex;gap:10px;justify-content:center;flex-wrap:wrap;}
  button{cursor:pointer;border:none;outline:none;font-family:inherit;}
  .btn{padding:10px 18px;border-radius:10px;font-size:12.5px;font-weight:700;letter-
spacing:0.5px;transition:transform .15s, box-shadow .15s;color:#fff;}
  .btn:hover{transform:translateY(-2px);}
  .btn:active{transform:translateY(0);}
  .btn-start{background:linear-gradient(135deg,#22c55e,#0ea55f);box-shadow:0 4px
14px rgba(34,197,94,0.35);}
  .btn-pause{background:linear-gradient(135deg,#ffb300,#ff8f00);box-shadow:0 4px
14px rgba(255,152,0,0.3);}
  .btn-reset{background:linear-gradient(135deg,#3b82f6,#1d4ed8);box-shadow:0 4px
14px rgba(59,130,246,0.3);}
  .btn-random{background:linear-gradient(135deg,#8b5cf6,#6d28d9);box-shadow:0
4px 14px rgba(139,92,246,0.3);}
  .btn-emergency{background:linear-gradient(135deg,#ff3b3b,#b71c1c);box-shadow:0
4px 14px rgba(255,59,59,0.4);}
  .btn-export{background:linear-gradient(135deg,#00bcd4,#0097a7);box-shadow:0
4px 14px rgba(0,188,212,0.3);}
  .btn:disabled{opacity:0.4;cursor:not-allowed;transform:none;}

  .emergency-banner{display:none;background:linear-
gradient(90deg,#7a0000,#b30000);color:#fff;text-align:center;padding:10px;border-
radius:10px;font-weight:800;letter-spacing:1px;margin:0 0
14px;animation:blinkBadge 0.8s infinite;}
  .emergency-banner.show{display:block;}

  /* ===== LAYOUT ===== */
  .main-layout{display:grid;grid-template-columns:290px 1fr 350px;gap:14px;align-
items:start;}
  @media (max-width:1250px){.main-layout{grid-template-columns:1fr;}}

  .panel{padding:14px 16px;margin-bottom:14px;}
  .panel h3{margin:0 0 10px;font-size:13px;letter-spacing:1px;color:var(--cyan);text-
transform:uppercase;border-bottom:1px solid var(--card-border);padding-
bottom:8px;}
  .field{margin-bottom:14px;}
  .field label{display:flex;justify-content:space-between;font-size:12px;color:var(--
```

```
dim);margin-bottom:6px;}
  .field label span.val{color:#fff;font-weight:700;}
  input[type=range]{width:100%;height:5px;border-
radius:4px;background:rgba(255,255,255,0.12);outline:none;-webkit-
appearance:none;}
  input[type=range]::-webkit-slider-thumb{-webkit-
appearance:none;width:15px;height:15px;border-radius:50%;background:var(--
cyan);box-shadow:0 0 8px rgba(0,229,255,0.7);cursor:pointer;}
  select{width:100%;padding:8px 10px;border-
radius:8px;background:#0d162c;color:#fff;border:1px solid var(--card-border);font-
size:12.5px;}
  .algo-desc{margin-top:10px;padding:10px
12px;background:rgba(0,229,255,0.06);border:1px solid rgba(0,229,255,0.2);border-
radius:10px;font-size:11.5px;color:var(--dim);}
  .algo-desc b{color:var(--cyan);display:block;margin-bottom:3px;font-size:12.5px;}

  .indicators{display:grid;grid-template-columns:1fr 1fr;gap:8px;font-
size:11.5px;color:var(--dim);}
  .indicators div{display:flex;align-items:center;gap:6px;}

  /* ===== SIMULATION AREA ===== */
  .sim-wrap{position:relative;}
  #simCanvas{width:100%;height:auto;aspect-ratio:1100/620;display:block;border-
radius:16px;background:linear-gradient(160deg,#060c1a,#0a1428);border:1px solid
var(--card-border);box-shadow:0 10px 40px rgba(0,0,0,0.5);cursor:crosshair;}
  .help-box{position:absolute;bottom:12px;left:12px;padding:10px 12px;font-
size:11px;color:var(--dim);max-width:230px;line-height:1.6;}
  .help-box b{color:var(--cyan);display:block;margin-bottom:4px;font-size:11.5px;}
  .details-panel{position:absolute;top:12px;right:12px;width:230px;padding:12px
14px;display:none;}
  .details-panel.hidden{display:none;}
  .details-panel:not(.hidden){display:block;}
  .panel-head{display:flex;justify-content:space-between;align-items:center;margin-
bottom:8px;border-bottom:1px solid var(--card-border);padding-bottom:6px;}
  .panel-head strong{color:var(--cyan);font-size:13px;}
  .close-btn{background:none;color:var(--dim);font-size:14px;padding:2px
6px;border-radius:6px;}
  .close-btn:hover{color:#fff;background:rgba(255,255,255,0.08);}
  .detail-row{display:flex;justify-content:space-between;font-size:11.5px;color:var(--
dim);margin-bottom:6px;}
  .detail-row b{color:#fff;}

  .mini-status{position:absolute;top:12px;left:12px;display:flex;gap:8px;flex-
wrap:wrap;max-width:60%;}
  .mini-status span{padding:5px 10px;border-radius:8px;font-size:10.5px;font-
```

```
weight:700;background:rgba(0,0,0,0.4);border:1px solid var(--card-border);}

  /* ===== RIGHT PANEL ===== */
  .metrics-grid{display:grid;grid-template-columns:1fr 1fr;gap:10px;}
  .metric-card{padding:12px;border-
radius:12px;background:rgba(255,255,255,0.04);border:1px solid var(--card-
border);text-align:center;}
  .metric-card .mv{font-size:20px;font-weight:800;color:#fff;}
  .metric-card .ml{font-size:10px;color:var(--dim);letter-spacing:0.5px;margin-
top:2px;text-transform:uppercase;}
  .metric-card.cyan .mv{color:var(--cyan);}
  .metric-card.green .mv{color:var(--green);}
  .metric-card.orange .mv{color:var(--orange);}
  .metric-card.red .mv{color:var(--red);}

  .row-stats{display:flex;justify-content:space-between;font-size:12px;color:var(--
dim);margin-bottom:8px;}
  .row-stats b{color:#fff;}

  table{width:100%;border-collapse:collapse;font-size:11px;}
  th{text-align:left;color:var(--dim);font-weight:600;padding:6px 4px;border-
bottom:1px solid var(--card-border);text-transform:uppercase;font-size:9.5px;}
  td{padding:6px 4px;border-bottom:1px solid rgba(255,255,255,0.04);color:#dce8fb;}
  .table-scroll{max-height:220px;overflow-y:auto;}
  .badge{padding:3px 8px;border-radius:6px;font-size:9.5px;font-weight:700;}
  .badge.ok{background:rgba(34,197,94,0.15);color:#7CFFB2;}
  .badge.warn{background:rgba(255,152,0,0.15);color:#ffcc80;}
  .badge.danger{background:rgba(255,59,59,0.18);color:#ff9a9a;}
  .badge.off{background:rgba(255,255,255,0.08);color:var(--dim);}
  .tag{padding:3px 7px;border-radius:6px;font-size:9.5px;font-weight:700;}
  .tag.moving{background:rgba(59,130,246,0.15);color:#a9c8ff;}
  .tag.arriving{background:rgba(34,197,94,0.15);color:#7CFFB2;}
  .tag.idle{background:rgba(255,255,255,0.08);color:var(--dim);}
  .batt-low{color:#ff8a8a !important;}
  .batt-mid{color:#ffcc80 !important;}
  .batt-ok{color:#7CFFB2 !important;}

  .chart-box{margin-bottom:10px;}
  .chart-box .clabel{font-size:10.5px;color:var(--dim);margin-bottom:4px;text-
transform:uppercase;letter-spacing:0.5px;}
  canvas.chart{width:100%;height:70px;border-
radius:8px;background:rgba(255,255,255,0.03);}

  #eventLog{max-height:180px;overflow-y:auto;font-size:11px;color:var(--dim);}
  .log-entry{padding:5px 0;border-bottom:1px solid rgba(255,255,255,0.04);font-
```

```
family:'Consolas',monospace;}
  .log-entry:first-child{color:#cfe8ff;}

  .info-line{font-size:12px;color:var(--dim);display:flex;justify-content:space-
between;padding:4px 0;}
  .info-line b{color:#fff;}

  /* ===== SECTIONS BELOW ===== */
  .section-title{font-size:20px;font-weight:800;color:#fff;margin:26px 0 14px;text-
align:center;letter-spacing:1px;}
  .section-title span{color:var(--cyan);}
  .grid3{display:grid;grid-template-columns:1fr 1fr 1fr;gap:14px;}
  @media (max-width:900px){.grid3{grid-template-columns:1fr;}}
  .overview-card{padding:18px;}
  .overview-card h4{margin:0 0 10px;color:var(--cyan);font-size:14px;}
  .overview-card ul{margin:0;padding-left:18px;color:var(--dim);font-size:12.5px;line-
height:1.8;}
  .overview-card p{color:var(--dim);font-size:12.5px;line-height:1.7;margin:0;}

  /* ARCHITECTURE DIAGRAM */
  .arch-flow{display:flex;flex-direction:column;align-
items:center;gap:4px;padding:22px;}
  .arch-box{padding:10px 22px;border-
radius:10px;background:rgba(0,229,255,0.08);border:1px solid
rgba(0,229,255,0.3);color:#cdf3ff;font-weight:700;font-size:12.5px;text-
align:center;letter-spacing:0.5px;}
  .arch-arrow{color:var(--dim2);font-size:16px;margin:2px 0;}
  .arch-agents{display:grid;grid-template-
columns:repeat(4,1fr);gap:6px;padding:10px;border:1px solid
rgba(255,255,255,0.15);border-
radius:10px;background:rgba(255,255,255,0.03);margin-top:2px;}
  .arch-agents span{background:rgba(59,130,246,0.15);border:1px solid
rgba(59,130,246,0.35);border-radius:6px;padding:6px 4px;text-align:center;font-
size:11px;color:#a9c8ff;font-weight:700;}

  /* FLOW DIAGRAM */
  .flow-steps{display:flex;flex-wrap:wrap;justify-
content:center;gap:8px;padding:16px;}
  .flow-step{padding:10px 16px;border-
radius:20px;background:rgba(139,92,246,0.1);border:1px solid
rgba(139,92,246,0.35);color:#e0d4ff;font-size:12px;font-
weight:600;display:flex;align-items:center;gap:6px;}
  .flow-step .num{background:#8b5cf6;color:#fff;width:18px;height:18px;border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:10px;}
  .flow-arrow-sm{color:var(--dim2);font-size:14px;}
```

```
  footer{text-align:center;padding:26px 10px;color:var(--dim);font-size:12px;border-top:1px solid var(--card-border);margin-top:30px;}
  footer .foot-title{color:#fff;font-weight:700;margin-bottom:6px;}
  footer .foot-tag{color:var(--cyan);font-size:11px;margin-top:8px;letter-spacing:0.5px;}
  footer .foot-brand{margin-top:14px;font-size:10.5px;color:var(--dim2);}
</style>
</head>
<body>
<div class="app">

  <!-- ============ TOP BAR ============ -->
  <div class="topbar">
    <div class="brand">
      <div class="logo">SR-MACS</div>
      <div class="subb">Swarm Robotics Multi-Agent<br>Coordination System</div>
    </div>
    <div class="center-title">
      <h1>SWARM ROBOTICS</h1>
      <div class="sub">MULTI-AGENT COORDINATION SYSTEM</div>
    </div>
    <div id="systemStatus" class="status-badge paused">□ SYSTEM READY</div>
  </div>

  <!-- ============ TITLE / CONTROLS ============ -->
  <div class="titlebar glass">
    <h2>Swarm Robotics Based Multi-Agent Coordination System</h2>
    <div class="small-sub">Autonomous Multi-Agent Coordination, Navigation & Task Allocation</div>
    <div class="ctrl-buttons">
      <button class="btn btn-start" id="btnStart">▸ START SIMULATION</button>
      <button class="btn btn-pause" id="btnPause">⏸ PAUSE</button>
      <button class="btn btn-reset" id="btnReset">↻ RESET</button>
      <button class="btn btn-random" id="btnRandom">🎲 RANDOMIZE</button>
      <button class="btn btn-emergency" id="btnEmergency">⛔ EMERGENCY STOP</button>
      <button class="btn btn-export" id="btnExport">⬇ EXPORT SIMULATION DATA</button>
    </div>
  </div>

  <div class="emergency-banner" id="emergencyBanner">🛑 EMERGENCY STOP ACTIVATED — ALL ROBOTS HALTED</div>
```

```
<!-- ============ MAIN LAYOUT ============ -->
<div class="main-layout">

  <!-- LEFT SIDEBAR -->
  <div class="sidebar">
    <div class="panel glass">
      <h3>Swarm Configuration</h3>
      <div class="field">
        <label>Number of Robots <span class="val" id="valRobots">8</span></label>
        <input type="range" id="sliderRobots" min="3" max="20" value="8" step="1">
      </div>
      <div class="field">
        <label>Number of Targets <span class="val" id="valTargets">3</span></label>
        <input type="range" id="sliderTargets" min="1" max="8" value="3" step="1">
      </div>
      <div class="field">
        <label>Simulation Speed <span class="val" id="valSpeed">1x</span></label>
        <input type="range" id="sliderSpeed" min="0" max="3" value="1" step="1">
      </div>
      <div class="field">
        <label>Communication Range <span class="val" id="valComm">150
px</span></label>
        <input type="range" id="sliderComm" min="50" max="250" value="150"
step="10">
      </div>
      <div class="field">
        <label>Obstacle Density <span class="val"
id="valObstacle">Medium</span></label>
        <input type="range" id="sliderObstacle" min="0" max="2" value="1"
step="1">
      </div>
    </div>

    <div class="panel glass">
      <h3>Coordination Algorithm</h3>
      <select id="selectAlgorithm">
        <option value="flocking">Flocking</option>
        <option value="leaderfollower">Leader-Follower</option>
        <option value="potentialfield">Potential Field</option>
        <option value="consensus">Consensus</option>
        <option value="nearest">Nearest Target Assignment</option>
        <option value="cooperative">Cooperative Navigation</option>
      </select>
      <div class="algo-desc" id="algoDesc"><b>Flocking</b>Robots maintain
```

```
separation, alignment and cohesion while moving toward their assigned targets.</div>
    </div>

    <div class="panel glass">
      <h3>Formation Mode</h3>
      <select id="selectFormation">
        <option value="free">Free Swarm</option>
        <option value="line">Line Formation</option>
        <option value="v">V Formation</option>
        <option value="circle">Circle Formation</option>
        <option value="grid">Grid Formation</option>
        <option value="leaderfollower">Leader-Follower</option>
      </select>
    </div>

    <div class="panel glass">
      <h3>Flocking Weights</h3>
      <div class="field"><label>Separation <span class="val"
id="valWSep">1.4</span></label><input type="range" id="wSep" min="0" max="3"
value="1.4" step="0.1"></div>
      <div class="field"><label>Alignment <span class="val"
id="valWAlign">1.0</span></label><input type="range" id="wAlign" min="0"
max="3" value="1.0" step="0.1"></div>
      <div class="field"><label>Cohesion <span class="val"
id="valWCoh">0.9</span></label><input type="range" id="wCoh" min="0"
max="3" value="0.9" step="0.1"></div>
      <div class="field"><label>Target Attraction <span class="val"
id="valWTarget">1.3</span></label><input type="range" id="wTarget" min="0"
max="3" value="1.3" step="0.1"></div>
      <div class="field"><label>Obstacle Avoidance <span class="val"
id="valWObs">1.7</span></label><input type="range" id="wObs" min="0" max="3"
value="1.7" step="0.1"></div>
    </div>

    <div class="panel glass">
      <h3>System Indicators</h3>
      <div class="indicators">
        <div>□ Navigation</div>
        <div>□ Communication</div>
        <div>□ Collision Avoidance</div>
        <div>□ Task Allocation</div>
        <div id="indFormation">◯ Formation Control</div>
        <div>□ Swarm Coordination</div>
      </div>
```

```
  </div>
 </div>

 <!-- MAIN SIMULATION -->
 <div class="sim-wrap">
  <canvas id="simCanvas" width="1100" height="620"></canvas>
  <div class="mini-status" id="miniStatus"></div>
  <div class="help-box glass">
   <b>INTERACTION</b>
   Click → Add Obstacle<br>
   Shift + Click → Add Target<br>
   Robot Click → Robot Details<br>
   Double-Click Target → Remove Target
  </div>
  <div class="details-panel glass hidden" id="robotDetailsPanel"></div>
 </div>

 <!-- RIGHT PANEL -->
 <div class="rightpanel">
  <div class="panel glass">
   <h3>Performance Metrics</h3>
   <div class="metrics-grid">
    <div class="metric-card cyan"><div class="mv"
id="metricActive">8</div><div class="ml">Active Robots</div></div>
    <div class="metric-card green"><div class="mv"
id="metricCompleted">0</div><div class="ml">Targets Completed</div></div>
    <div class="metric-card red"><div class="mv"
id="metricCollisions">0</div><div class="ml">Collisions</div></div>
    <div class="metric-card orange"><div class="mv"
id="metricObstacles">12</div><div class="ml">Obstacles</div></div>
    <div class="metric-card cyan"><div class="mv"
id="metricCommLinks">0</div><div class="ml">Comm Links</div></div>
    <div class="metric-card green"><div class="mv"
id="metricEfficiency">90%</div><div class="ml">Swarm Efficiency</div></div>
    <div class="metric-card cyan"><div class="mv" id="metricAvgSpeed">0
m/s</div><div class="ml">Average Speed</div></div>
    <div class="metric-card orange"><div class="mv"
id="metricCoverage">0%</div><div class="ml">Coverage</div></div>
   </div>
   <div class="row-stats" style="margin-top:10px;"><span>Collision
Avoidance</span><b style="color:#7CFFB2;">ACTIVE</b></div>
  </div>

  <div class="panel glass">
   <h3>Communication Network</h3>
```

```
      <div class="info-line"><span>Connected Robots</span><b
id="commConnected">0</b></div>
      <div class="info-line"><span>Messages / sec</span><b
id="commMsgRate">0</b></div>
      <div class="info-line"><span>Avg. Comm. Distance</span><b
id="commAvgDist">0 px</b></div>
      <div class="info-line"><span>Network Connectivity</span><b
id="commConnectivity">0%</b></div>
      <div id="leaderPanel" class="info-line" style="display:none;"><span>👑
LEADER</span><b id="leaderValue">R1</b></div>
      <div id="consensusPanel" class="info-line" style="display:none;"><span>
CONSENSUS LEVEL</span><b id="consensusValue">0%</b></div>
    </div>

    <div class="panel glass">
      <h3>Task Allocation</h3>
      <div class="table-scroll">
        <table>

<thead><tr><th>Robot</th><th>Task</th><th>Distance</th><th>Status</th></tr></t
head>
          <tbody id="taskTableBody"></tbody>
        </table>
      </div>
    </div>

    <div class="panel glass">
      <h3>Robot Status Monitor</h3>
      <div class="table-scroll">
        <table>

<thead><tr><th>ID</th><th>X</th><th>Y</th><th>Spd</th><th>Tgt</th><th>Batt
</th><th>Status</th></tr></thead>
          <tbody id="robotTableBody"></tbody>
        </table>
      </div>
    </div>

    <div class="panel glass">
      <h3>Real-Time Charts</h3>
      <div class="chart-box"><div class="clabel">Swarm Efficiency
(%)</div><canvas class="chart" id="chartEfficiency"></canvas></div>
      <div class="chart-box"><div class="clabel">Active Robots</div><canvas
class="chart" id="chartActive"></canvas></div>
      <div class="chart-box"><div class="clabel">Completed Tasks</div><canvas
```

```
class="chart" id="chartCompleted"></canvas></div>
      <div class="chart-box"><div class="clabel">Collision Count</div><canvas
class="chart" id="chartCollisions"></canvas></div>
    </div>

    <div class="panel glass">
      <h3>Event Log</h3>
      <div id="eventLog"></div>
    </div>
  </div>
</div>

<!-- ============ PROJECT OVERVIEW ============ -->
<div class="section-title">PROJECT <span>OVERVIEW</span></div>
<div class="grid3">
  <div class="overview-card glass">
    <h4>Objective</h4>
    <p>To develop an autonomous multi-agent coordination system where multiple
robots cooperate to navigate an environment, avoid obstacles, communicate, and
complete assigned tasks efficiently.</p>
  </div>
  <div class="overview-card glass">
    <h4>Key Technologies</h4>
    <ul>
      <li>Multi-Agent Systems</li>
      <li>Swarm Intelligence</li>
      <li>Artificial Intelligence</li>
      <li>Autonomous Robotics</li>
      <li>Distributed Coordination</li>
      <li>Collision Avoidance</li>
      <li>Task Allocation</li>
      <li>HTML5 Canvas & JavaScript</li>
    </ul>
  </div>
  <div class="overview-card glass">
    <h4>Applications</h4>
    <ul>
      <li>Search and Rescue</li>
      <li>Agriculture</li>
      <li>Military Surveillance</li>
      <li>Warehouse Automation</li>
      <li>Environmental Monitoring</li>
      <li>Disaster Management</li>
      <li>Smart Transportation</li>
      <li>Industrial Automation</li>
```

```
    </ul>
   </div>
  </div>

  <!-- ============ ARCHITECTURE ============ -->
  <div class="section-title">SYSTEM <span>ARCHITECTURE</span></div>
  <div class="glass arch-flow">
   <div class="arch-box">USER</div>
   <div class="arch-arrow">↓</div>
   <div class="arch-box">CONTROL PANEL</div>
   <div class="arch-arrow">↓</div>
   <div class="arch-box">SWARM COORDINATION ENGINE</div>
   <div class="arch-arrow">↓</div>
   <div class="arch-box">MULTI-AGENT DECISION MAKING</div>
   <div class="arch-arrow">↓</div>
   <div class="arch-agents">
    <span>R1</span><span>R2</span><span>R3</span><span>R4</span>
    <span>R5</span><span>R6</span><span>R7</span><span>R8</span>
   </div>
   <div class="arch-arrow">↓</div>
   <div class="arch-box">COMMUNICATION NETWORK</div>
   <div class="arch-arrow">↓</div>
   <div class="arch-box">ENVIRONMENT</div>
   <div class="arch-arrow">↓</div>
   <div class="arch-box">TARGETS + OBSTACLES</div>
  </div>

  <!-- ============ ALGORITHM FLOW ============ -->
  <div class="section-title">SWARM COORDINATION <span>FLOW</span></div>
  <div class="glass flow-steps">
   <div class="flow-step"><span class="num">1</span>Initialize Robots</div><span
class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">2</span>Detect
Environment</div><span class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">3</span>Detect
Neighbours</div><span class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">4</span>Assign Targets</div><span
class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">5</span>Calculate Swarm
Forces</div><span class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">6</span>Avoid Obstacles</div><span
class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">7</span>Move Robots</div><span
class="flow-arrow-sm">→</span>
   <div class="flow-step"><span class="num">8</span>Check Target
```

```
Completion</div><span class="flow-arrow-sm">→</span>
    <div class="flow-step"><span class="num">9</span>Update
Communication</div><span class="flow-arrow-sm">→</span>
    <div class="flow-step"><span class="num">10</span>Update Metrics</div><span
class="flow-arrow-sm">→</span>
    <div class="flow-step"><span class="num">↻</span>Repeat</div>
  </div>

  <footer>
    <div class="foot-title">Swarm Robotics Based Multi-Agent Coordination
System</div>
    <div>Autonomous Coordination • Intelligent Navigation • Distributed Decision
Making</div>
    <div class="foot-tag">AI & ROBOTICS SIMULATION PLATFORM</div>
    <div class="foot-brand">FINAL YEAR CAPSTONE PROJECT — SR-
MACS</div>
  </footer>

</div>

<script>
(function(){
"use strict";

/* ================ CONSTANTS ================ */
var WIDTH = 1100, HEIGHT = 620;
var POS_MULT = 1.5, MAX_SPEED = 2.2, MAX_SPEED_LOW = 1.1;
var COLLISION_DIST = 18, SEP_RADIUS = 45, TARGET_HIT = 16,
TARGET_NEAR = 90;

var ALGO_DESC = {
  flocking:{name:"Flocking", text:"Robots maintain separation, alignment and
cohesion while moving toward their assigned targets."},
  leaderfollower:{name:"Leader-Follower", text:"One robot is designated leader and
moves toward the target. Other robots follow while keeping a relative formation
offset."},
  potentialfield:{name:"Potential Field", text:"Targets act as attractive potentials and
obstacles/robots as repulsive potentials; robots move along the combined force
field."},
  consensus:{name:"Consensus", text:"Robots exchange velocity information with
neighbours and gradually converge toward a shared heading and swarm state."},
  nearest:{name:"Nearest Target Assignment", text:"Each robot greedily selects and
moves directly toward its nearest available target using only local collision
avoidance."},
  cooperative:{name:"Cooperative Navigation", text:"Robots communicate to divide
```

```
tasks by distance and workload, balancing flocking behaviour with obstacle-aware
navigation."}
};

/* ================ STATE ================ */
var state = {
  robots:[], targets:[], obstacles:[], commEdges:[],
  algorithm:"flocking", formation:"free",
  config:{numRobots:8, numTargets:3, obstacleDensity:"medium"},
  weights:{sep:1.4, align:1.0, coh:0.9, target:1.3, obstacle:1.7},
  commRange:150, speed:1,
  running:false, emergency:false,
  metrics:{collisions:0, completed:0},
  charts:{efficiency:[], active:[], completed:[], collisions:[]},
  coverageGrid:[], targetCounter:1,
  leaderId:null, selectedRobotId:null,
  eventLog:[], startTime:null,
  formationCenter:{x:WIDTH/2, y:HEIGHT/2}
};

/* ================ UTIL ================ */
function rand(a,b){return a+Math.random()*(b-a);}
function dist(a,b){return Math.hypot(a.x-b.x,a.y-b.y);}
function randPos(margin){ margin=margin||60; return {x:rand(margin,WIDTH-
margin), y:rand(margin,HEIGHT-margin)}; }
function safeRandomPosition(){
  for(var i=0;i<30;i++){
    var p = randPos(60); var ok=true;
    for(var j=0;j<state.obstacles.length;j++){ var o=state.obstacles[j]; if(dist(p,o) <
o.r+42){ ok=false; break; } }
    if(ok) return p;
  }
  return randPos(60);
}
function getRobotById(id){ for(var i=0;i<state.robots.length;i++){
if(state.robots[i].id===id) return state.robots[i]; } return null; }
function getAssignedTarget(r){ if(!r.targetId) return null; for(var
i=0;i<state.targets.length;i++){ if(state.targets[i].id===r.targetId) return
state.targets[i]; } return null; }
function normalizeVec(v){ var m=Math.hypot(v.x,v.y); if(m>0){ v.x/=m; v.y/=m; }
return v; }
function escapeHtml(s){ return
s.replace(/&/g,"&amp;").replace(/</g,"&lt;").replace(/>/g,"&gt;"); }

/* ================ EVENT LOG ================ */
```

```
function logEvent(msg){
  var t = new Date().toLocaleTimeString("en-GB",{hour12:false});
  state.eventLog.unshift("["+t+"] "+msg);
  if(state.eventLog.length>50) state.eventLog.pop();
  renderEventLog();
}
function renderEventLog(){
  var el = document.getElementById("eventLog");
  var html = "";
  for(var i=0;i<state.eventLog.length;i++){ html += '<div class="log-
entry">'+escapeHtml(state.eventLog[i])+'</div>'; }
  el.innerHTML = html;
}

/* ================ INITIALIZATION ================ */
function regenerateObstacles(){
  state.obstacles = [];
  var densityMap = {low:6, medium:12, high:20};
  var count = densityMap[state.config.obstacleDensity];
  for(var i=0;i<count;i++){
    var p = randPos(50);
    state.obstacles.push({x:p.x, y:p.y, r: 14+Math.random()*16});
  }
}

function initSimulation(){
  regenerateObstacles();
  state.targets = [];
  state.targetCounter = 1;
  for(var i=0;i<state.config.numTargets;i++){
    var p = safeRandomPosition();
    state.targets.push({id:"T"+(state.targetCounter++), x:p.x, y:p.y,
status:"WAITING"});
  }
  state.robots = [];
  for(var k=1;k<=state.config.numRobots;k++){
    var pr = safeRandomPosition();
    state.robots.push({
      id:"R"+k, x:pr.x, y:pr.y,
      vx:(Math.random()-0.5)*1.5, vy:(Math.random()-0.5)*1.5,
      battery: 85+Math.random()*15, status:"ACTIVE", targetId:null, collisionFlash:0
    });
  }
  state.leaderId = state.robots.length ? state.robots[0].id : null;
  state.metrics = {collisions:0, completed:0};
```

```
  state.coverageGrid = new Array(22*13).fill(false);
  state.charts = {efficiency:[], active:[], completed:[], collisions:[]};
  state.commEdges = [];
  state.selectedRobotId = null;
  document.getElementById("robotDetailsPanel").classList.add("hidden");
  state.eventLog = [];
  state.startTime = performance.now();
  logEvent("SYSTEM INITIALIZED");
  logEvent(state.robots.length+" robots deployed, "+state.targets.length+" targets,
"+state.obstacles.length+" obstacles");
  assignTargets();
  refreshPanels();
  render();
}

/* ================= DYNAMIC ADJUSTMENTS ================= */
function adjustRobotCount(n){
  var cur = state.robots.length;
  if(n>cur){
    for(var i=cur+1;i<=n;i++){
      var p = safeRandomPosition();
      state.robots.push({id:"R"+i, x:p.x, y:p.y, vx:(Math.random()-0.5)*1.5,
vy:(Math.random()-0.5)*1.5, battery:100, status:"ACTIVE", targetId:null,
collisionFlash:0});
    }
  } else if(n<cur){
    state.robots.splice(n);
    if(state.leaderId && !getRobotById(state.leaderId) && state.robots.length)
state.leaderId = state.robots[0].id;
  }
  assignTargets();
  logEvent("Swarm size adjusted to "+n+" robots");
}
function adjustTargetCount(n){
  var cur = state.targets.length;
  if(n>cur){
    for(var i=cur;i<n;i++){
      var p = safeRandomPosition();
      state.targets.push({id:"T"+(state.targetCounter++), x:p.x, y:p.y,
status:"WAITING"});
    }
  } else if(n<cur){
    state.targets.splice(n);
  }
  assignTargets();
```

```
  logEvent("Target count adjusted to "+n);
}

/* ================ TASK ALLOCATION ================ */
function assignTargets(){
  var crowd = {};
  for(var i=0;i<state.targets.length;i++) crowd[state.targets[i].id]=0;
  for(var r=0;r<state.robots.length;r++){
    var rob = state.robots[r];
    var t = getAssignedTarget(rob);
    if(t && t.status!=="COMPLETED") crowd[t.id] = (crowd[t.id]||0)+1;
  }
  for(var k=0;k<state.robots.length;k++){
    var robot = state.robots[k];
    if(robot.battery<=0) continue;
    var cur = getAssignedTarget(robot);
    if(cur && cur.status!=="COMPLETED") continue;
    var best=null, bestCost=Infinity;
    for(var j=0;j<state.targets.length;j++){
      var tg = state.targets[j];
      if(tg.status==="COMPLETED") continue;
      var d = dist(tg, robot);
      var penalty = (state.algorithm==="nearest") ? 0 : (crowd[tg.id]||0)*40;
      var cost = d+penalty;
      if(cost<bestCost){ bestCost=cost; best=tg; }
    }
    if(best){
      robot.targetId = best.id;
      crowd[best.id] = (crowd[best.id]||0)+1;
      if(best.status==="WAITING") best.status="ASSIGNED";
      logEvent(robot.id+" assigned to "+best.id);
    }
  }
}

/* ================ FORMATION LOGIC ================ */
function updateFormationCenter(){
  if(state.robots.length===0) return;
  var cx=0, cy=0;
  for(var i=0;i<state.robots.length;i++){ cx+=state.robots[i].x; cy+=state.robots[i].y; }
  cx/=state.robots.length; cy/=state.robots.length;
  state.formationCenter.x += (cx-state.formationCenter.x)*0.02;
  state.formationCenter.y += (cy-state.formationCenter.y)*0.02;
}
function getFollowerOffset(r){
```

```
  var leader = getRobotById(state.leaderId);
  if(!leader) return {x:0,y:0};
  var followers = state.robots.filter(function(x){ return x.id!==state.leaderId; });
  var idx = followers.indexOf(r);
  var col = (idx%3)-1, row = Math.floor(idx/3)+1;
  var ang = Math.atan2(leader.vy, leader.vx);
  var speed = Math.hypot(leader.vx,leader.vy);
  if(speed<0.05) ang = 0;
  var fx=Math.cos(ang), fy=Math.sin(ang);
  var rx=-fy, ry=fx;
  return { x: -fx*row*45 + rx*col*40, y: -fy*row*45 + ry*col*40 };
}
function getFormationSlot(r){
  var c = state.formationCenter;
  var idx = state.robots.indexOf(r);
  var n = state.robots.length;
  if(state.formation==="line"){
    return {x: c.x + (idx-(n-1)/2)*40, y: c.y};
  } else if(state.formation==="v"){
    if(idx===0) return {x:c.x, y:c.y};
    var rank = Math.ceil(idx/2), side = (idx%2===1)?1:-1;
    return {x: c.x - rank*35, y: c.y + side*rank*25};
  } else if(state.formation==="circle"){
    var angle = (idx/Math.max(n,1))*Math.PI*2;
    return {x: c.x+Math.cos(angle)*120, y: c.y+Math.sin(angle)*120};
  } else if(state.formation==="grid"){
    var cols = Math.ceil(Math.sqrt(n));
    var col = idx%cols, row = Math.floor(idx/cols);
    return {x: c.x+(col-(cols-1)/2)*45, y: c.y+(row-1)*45};
  } else if(state.formation==="leaderfollower"){
    if(r.id===state.leaderId) return null;
    var leader = getRobotById(state.leaderId);
    if(!leader) return null;
    var off = getFollowerOffset(r);
    return {x: leader.x+off.x, y: leader.y+off.y};
  }
  return null;
}

/* ================ FORCE CALCULATION ================ */
function computeForces(r){
  var sep={x:0,y:0}, align={x:0,y:0}, coh={x:0,y:0}, tgt={x:0,y:0}, obs={x:0,y:0},
bound={x:0,y:0}, form={x:0,y:0};
  var neighborCount=0;
  for(var i=0;i<state.robots.length;i++){
```

```
  var other = state.robots[i];
  if(other===r) continue;
  var dx=r.x-other.x, dy=r.y-other.y, d=Math.hypot(dx,dy);
  if(d<state.commRange && d>0){
    neighborCount++;
    align.x+=other.vx; align.y+=other.vy;
    coh.x+=other.x; coh.y+=other.y;
  }
  if(d<SEP_RADIUS && d>0){
    var f=(SEP_RADIUS-d)/SEP_RADIUS;
    sep.x += (dx/d)*f; sep.y += (dy/d)*f;
  }
}
if(neighborCount>0){
  align.x/=neighborCount; align.y/=neighborCount;
  align.x-=r.vx; align.y-=r.vy;
  coh.x = coh.x/neighborCount - r.x;
  coh.y = coh.y/neighborCount - r.y;
  normalizeVec(coh);
}
normalizeVec(sep); normalizeVec(align);

var t = getAssignedTarget(r);
if(t){ tgt.x=t.x-r.x; tgt.y=t.y-r.y; normalizeVec(tgt); }

for(var o=0;o<state.obstacles.length;o++){
  var ob = state.obstacles[o];
  var odx=r.x-ob.x, ody=r.y-ob.y, od=Math.hypot(odx,ody);
  var threat = ob.r+35;
  if(od<threat && od>0){
    var of=(threat-od)/threat;
    obs.x += (odx/od)*of*2; obs.y += (ody/od)*of*2;
  }
}

var margin=40;
if(r.x<margin) bound.x += (margin-r.x)/margin;
if(r.x>WIDTH-margin) bound.x -= (r.x-(WIDTH-margin))/margin;
if(r.y<margin) bound.y += (margin-r.y)/margin;
if(r.y>HEIGHT-margin) bound.y -= (r.y-(HEIGHT-margin))/margin;

if(state.formation!=="free"){
  var desired = getFormationSlot(r);
  if(desired){ form.x = desired.x-r.x; form.y = desired.y-r.y; normalizeVec(form); }
}
```

```
  return {sep:sep, align:align, coh:coh, tgt:tgt, obs:obs, bound:bound, form:form,
neighborCount:neighborCount};
}

function applyAlgorithm(r, f){
  var w = state.weights;
  var ax=0, ay=0;
  switch(state.algorithm){
    case "flocking":
      ax = f.sep.x*w.sep + f.align.x*w.align + f.coh.x*w.coh + f.tgt.x*w.target +
f.obs.x*w.obstacle;
      ay = f.sep.y*w.sep + f.align.y*w.align + f.coh.y*w.coh + f.tgt.y*w.target +
f.obs.y*w.obstacle;
      break;
    case "leaderfollower":
      if(r.id===state.leaderId){
        ax = f.tgt.x*1.6 + f.obs.x*w.obstacle + f.sep.x*w.sep;
        ay = f.tgt.y*1.6 + f.obs.y*w.obstacle + f.sep.y*w.sep;
      } else {
        var leader = getRobotById(state.leaderId);
        if(leader){
          var off = getFollowerOffset(r);
          var dx=(leader.x+off.x)-r.x, dy=(leader.y+off.y)-r.y;
          var dn={x:dx,y:dy}; normalizeVec(dn);
          ax = dn.x*1.8 + f.sep.x*w.sep*1.2 + f.obs.x*w.obstacle;
          ay = dn.y*1.8 + f.sep.y*w.sep*1.2 + f.obs.y*w.obstacle;
        } else {
          ax = f.tgt.x*w.target; ay = f.tgt.y*w.target;
        }
      }
      break;
    case "potentialfield":
      ax = f.tgt.x*w.target*1.3 + f.obs.x*w.obstacle*1.6 + f.sep.x*w.sep*1.4;
      ay = f.tgt.y*w.target*1.3 + f.obs.y*w.obstacle*1.6 + f.sep.y*w.sep*1.4;
      break;
    case "consensus":
      ax = f.align.x*2.2 + f.coh.x*0.8 + f.tgt.x*0.5 + f.sep.x*w.sep + f.obs.x*w.obstacle;
      ay = f.align.y*2.2 + f.coh.y*0.8 + f.tgt.y*0.5 + f.sep.y*w.sep + f.obs.y*w.obstacle;
      break;
    case "nearest":
      ax = f.tgt.x*2.0 + f.sep.x*w.sep*1.5 + f.obs.x*w.obstacle;
      ay = f.tgt.y*2.0 + f.sep.y*w.sep*1.5 + f.obs.y*w.obstacle;
      break;
    case "cooperative":
```

```
      ax = f.sep.x*w.sep + f.align.x*w.align*0.6 + f.coh.x*w.coh*0.6 +
f.tgt.x*w.target*1.3 + f.obs.x*w.obstacle;
      ay = f.sep.y*w.sep + f.align.y*w.align*0.6 + f.coh.y*w.coh*0.6 +
f.tgt.y*w.target*1.3 + f.obs.y*w.obstacle;
      break;
  }
  ax += f.bound.x*3 + f.form.x*(state.formation!=="free"?1.6:0);
  ay += f.bound.y*3 + f.form.y*(state.formation!=="free"?1.6:0);
  return {ax:ax, ay:ay};
}

/* ================ PHYSICS UPDATE ================ */
function updateSimulation(dt){
  updateFormationCenter();
  computeCommunicationEdges();

  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    if(r.battery<=0){ r.vx=0; r.vy=0; continue; }
    var f = computeForces(r);
    var a = applyAlgorithm(r, f);
    r.vx = r.vx*0.88 + a.ax*0.12;
    r.vy = r.vy*0.88 + a.ay*0.12;
    var sp = Math.hypot(r.vx, r.vy);
    var maxSp = r.battery<20 ? MAX_SPEED_LOW : MAX_SPEED;
    if(sp>maxSp){ r.vx = r.vx/sp*maxSp; r.vy = r.vy/sp*maxSp; }
    r.x += r.vx*dt*POS_MULT;
    r.y += r.vy*dt*POS_MULT;
    r.x = Math.max(14, Math.min(WIDTH-14, r.x));
    r.y = Math.max(14, Math.min(HEIGHT-14, r.y));
    if(r.collisionFlash>0) r.collisionFlash = Math.max(0, r.collisionFlash-dt);
  }

  resolveObstacleCollisions();
  detectCollisions();
  checkTargetCompletion();
  updateBattery(dt);
  markCoverage();
}

function resolveObstacleCollisions(){
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    for(var j=0;j<state.obstacles.length;j++){
      var o = state.obstacles[j];
```

```
      var dx=r.x-o.x, dy=r.y-o.y, d=Math.hypot(dx,dy);
      var minD=o.r+10;
      if(d<minD && d>0){
        var overlap = minD-d;
        r.x += (dx/d)*overlap;
        r.y += (dy/d)*overlap;
      }
    }
  }
}

function detectCollisions(){
  var n = state.robots.length;
  for(var i=0;i<n;i++){
    for(var j=i+1;j<n;j++){
      var a = state.robots[i], b = state.robots[j];
      if(a.battery<=0 || b.battery<=0) continue;
      var dx=b.x-a.x, dy=b.y-a.y, d=Math.hypot(dx,dy);
      if(d<COLLISION_DIST){
        var overlap = (COLLISION_DIST-d)||0.01;
        var nx = d>0?dx/d:1, ny = d>0?dy/d:0;
        a.x -= nx*overlap*0.5; a.y -= ny*overlap*0.5;
        b.x += nx*overlap*0.5; b.y += ny*overlap*0.5;
        if(a.collisionFlash<=0 && b.collisionFlash<=0){
          state.metrics.collisions++;
          logEvent("Collision detected between "+a.id+" and "+b.id);
        }
        a.collisionFlash = 1.2; b.collisionFlash = 1.2;
        a.vx -= nx*0.5; a.vy -= ny*0.5;
        b.vx += nx*0.5; b.vy += ny*0.5;
      }
    }
  }
}

function checkTargetCompletion(){
  for(var i=0;i<state.targets.length;i++){
    var t = state.targets[i];
    if(t.status==="COMPLETED") continue;
    var minDist = Infinity;
    for(var j=0;j<state.robots.length;j++){
      var r = state.robots[j];
      if(r.targetId===t.id){
        var d = dist(t,r);
        if(d<minDist) minDist=d;
```

```
        if(d<TARGET_HIT){
          t.status="COMPLETED";
          t.completedAt = performance.now();
          state.metrics.completed++;
          logEvent(t.id+" completed by "+r.id);
          r.targetId=null;
        }
      }
    }
    if(t.status!=="COMPLETED"){
      t.status = minDist<TARGET_NEAR ? "IN PROGRESS" :
(t.status==="WAITING"?"WAITING":"ASSIGNED");
    }
  }
  var kept = [];
  for(var k=0;k<state.targets.length;k++){
    var tg = state.targets[k];
    if(tg.status==="COMPLETED" && performance.now()-tg.completedAt>1600)
continue;
    kept.push(tg);
  }
  state.targets = kept;
  while(state.targets.length < state.config.numTargets){
    var p = safeRandomPosition();
    state.targets.push({id:"T"+(state.targetCounter++), x:p.x, y:p.y,
status:"WAITING"});
  }
}

function updateBattery(dt){
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    if(r.battery>0){
      var sp = Math.hypot(r.vx,r.vy);
      r.battery -= dt*(0.02 + sp*0.01);
      if(r.battery<0) r.battery=0;
    }
    if(r.battery<=0){ r.status="OFFLINE"; r.vx=0; r.vy=0; }
    else if(r.collisionFlash>0.4){ r.status="COLLISION WARNING"; }
    else if(r.battery<20){ r.status="LOW BATTERY"; }
    else { r.status="ACTIVE"; }
  }
}

function markCoverage(){
```

```
  var cellsX=22, cellsY=13;
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    var cx = Math.min(cellsX-1, Math.max(0, Math.floor(r.x/WIDTH*cellsX)));
    var cy = Math.min(cellsY-1, Math.max(0, Math.floor(r.y/HEIGHT*cellsY)));
    state.coverageGrid[cy*cellsX+cx] = true;
  }
}
function computeCoverage(){
  var visited=0;
  for(var i=0;i<state.coverageGrid.length;i++){ if(state.coverageGrid[i]) visited++; }
  return Math.round(visited/state.coverageGrid.length*100);
}

/* ================ COMMUNICATION ================ */
function computeCommunicationEdges(){
  var edges=[];
  var n = state.robots.length;
  for(var i=0;i<n;i++){
    for(var j=i+1;j<n;j++){
      var a=state.robots[i], b=state.robots[j];
      var d = dist(a,b);
      if(d<=state.commRange) edges.push({a:a,b:b,d:d});
    }
  }
  state.commEdges = edges;
}
function computeConnectivity(){
  var n = state.robots.length;
  if(n===0) return 0;
  var parent = []; for(var i=0;i<n;i++) parent.push(i);
  function find(x){ while(parent[x]!==x){ x=parent[x]; } return x; }
  function union(a,b){ var ra=find(a), rb=find(b); if(ra!==rb) parent[ra]=rb; }
  for(var e=0;e<state.commEdges.length;e++){
    var edge = state.commEdges[e];
    var ia = state.robots.indexOf(edge.a), ib = state.robots.indexOf(edge.b);
    union(ia,ib);
  }
  var counts={};
  for(var k=0;k<n;k++){ var root=find(k); counts[root]=(counts[root]||0)+1; }
  var maxComp = 0;
  for(var key in counts){ if(counts[key]>maxComp) maxComp=counts[key]; }
  return maxComp/n*100;
}
```

```
function reassignLeaderIfNeeded(){
  var leader = getRobotById(state.leaderId);
  if(!leader || leader.battery<=0){
    var candidates = state.robots.filter(function(r){ return r.battery>0;
}).sort(function(a,b){ return b.battery-a.battery; });
    if(candidates.length && candidates[0].id!==state.leaderId){
      state.leaderId = candidates[0].id;
      logEvent(candidates[0].id+" promoted to swarm leader");
    }
  }
}

/* ================ METRICS ================ */
function computeEfficiency(){
  var base = 90;
  base += state.metrics.completed*1.5;
  base -= state.metrics.collisions*2.5;
  var lowBatt = state.robots.filter(function(r){ return r.battery<20; }).length;
  base -= lowBatt*1.5;
  return Math.max(35, Math.min(99, Math.round(base)));
}

/* ================ UI REFRESH ================ */
function refreshPanels(){
  var activeRobots = state.robots.filter(function(r){ return r.battery>0; }).length;
  var edges = state.commEdges.length;
  var avgDist = edges ? (state.commEdges.reduce(function(s,e){ return s+e.d;
},0)/edges).toFixed(0) : 0;
  reassignLeaderIfNeeded();
  var connectivity = computeConnectivity().toFixed(0);
  var messagesPerSec = Math.round(edges*3.2 + Math.random()*4);

  document.getElementById("metricActive").textContent = activeRobots;
  document.getElementById("metricCompleted").textContent =
state.metrics.completed;
  document.getElementById("metricCollisions").textContent = state.metrics.collisions;
  document.getElementById("metricObstacles").textContent = state.obstacles.length;
  document.getElementById("metricCommLinks").textContent = edges;
  document.getElementById("metricEfficiency").textContent =
computeEfficiency()+"%";
  var avgSpeed = state.robots.length ? (state.robots.reduce(function(s,r){ return
s+Math.hypot(r.vx,r.vy); },0)/state.robots.length*1.1).toFixed(1) : "0.0";
  document.getElementById("metricAvgSpeed").textContent = avgSpeed+" m/s";
  document.getElementById("metricCoverage").textContent =
computeCoverage()+"%";
```

```
  document.getElementById("commConnected").textContent =
state.robots.filter(function(r){
    return state.commEdges.some(function(e){ return e.a===r || e.b===r; });
  }).length;
  document.getElementById("commMsgRate").textContent = messagesPerSec;
  document.getElementById("commAvgDist").textContent = avgDist+" px";
  document.getElementById("commConnectivity").textContent = connectivity+"%";

  var leaderPanel = document.getElementById("leaderPanel");
  if(state.algorithm==="leaderfollower" || state.formation==="leaderfollower"){
    leaderPanel.style.display="flex";
    document.getElementById("leaderValue").textContent = state.leaderId || "-";
  } else { leaderPanel.style.display="none"; }

  var consensusPanel = document.getElementById("consensusPanel");
  if(state.algorithm==="consensus"){
    consensusPanel.style.display="flex";
    var sx=0, sy=0;
    for(var i=0;i<state.robots.length;i++){
      var r = state.robots[i];
      var s = Math.hypot(r.vx,r.vy)||0.001;
      sx += r.vx/s; sy += r.vy/s;
    }
    var mag = state.robots.length ? Math.hypot(sx,sy)/state.robots.length : 0;
    document.getElementById("consensusValue").textContent =
Math.round(Math.min(100,mag*100))+"%";
  } else { consensusPanel.style.display="none"; }

  document.getElementById("indFormation").innerHTML = (state.formation!=="free"
? "□" : "◯") + " Formation Control";

  renderTaskTable();
  renderRobotTable();
  updateRobotDetailsPanel();
  renderMiniStatus();
}

function renderMiniStatus(){
  var el = document.getElementById("miniStatus");
  var html = "";
  var completed = state.targets.filter(function(t){ return t.status==="COMPLETED";
}).length;
  html += '<span style="color:#7CFFB2;">🎯 '+(state.config.numTargets)+'
TARGETS</span>';
```

```
  html += '<span style="color:#a9c8ff;">📡 '+state.commEdges.length+'
LINKS</span>';
  if(state.formation==="leaderfollower" || state.algorithm==="leaderfollower") html +=
'<span style="color:#ffd54a;">👑 LEADER: '+(state.leaderId||"-")+'</span>';
  el.innerHTML = html;
}

function renderTaskTable(){
  var tbody = document.getElementById("taskTableBody");
  var html = "";
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    var t = getAssignedTarget(r);
    var d = t ? dist(t,r).toFixed(0) : "-";
    var statusLabel = r.battery<=0 ? "OFFLINE" : (t ? (parseFloat(d)<20 ?
"ARRIVING" : "MOVING") : "IDLE");
    var cls = statusLabel==="ARRIVING" ? "arriving" : (statusLabel==="MOVING" ?
"moving" : "idle");
    html += "<tr><td>"+r.id+"</td><td>"+(t?t.id:"—")+"</td><td>"+(t?d+" px":"—
")+"</td><td><span class='tag "+cls+"'>"+statusLabel+"</span></td></tr>";
  }
  tbody.innerHTML = html;
}

function renderRobotTable(){
  var tbody = document.getElementById("robotTableBody");
  var html = "";
  var map = {"ACTIVE":"ok","LOW BATTERY":"warn","COLLISION
WARNING":"danger","OFFLINE":"off"};
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    var speed = Math.hypot(r.vx,r.vy).toFixed(1);
    var t = getAssignedTarget(r);
    var battClass = r.battery<20 ? "batt-low" : (r.battery<50 ? "batt-mid" : "batt-ok");
    var badgeCls = map[r.status] || "ok";
    var statusText = r.status==="LOW BATTERY" ? "⚠ LOW BATTERY" : r.status;
    html +=
"<tr><td>"+r.id+"</td><td>"+Math.round(r.x)+"</td><td>"+Math.round(r.y)+"</td>
<td>"+speed+"</td><td>"+(t?t.id:"-")+"</td><td
class="'+battClass+'">"+Math.round(r.battery)+"%</td><td><span class='badge
"+badgeCls+'">"+statusText+"</span></td></tr>";
  }
  tbody.innerHTML = html;
}
```

```
function updateRobotDetailsPanel(){
  if(!state.selectedRobotId) return;
  var r = getRobotById(state.selectedRobotId);
  var panel = document.getElementById("robotDetailsPanel");
  if(!r){ panel.classList.add("hidden"); return; }
  var speed = (Math.hypot(r.vx,r.vy)*1.1).toFixed(2);
  var dir = (Math.atan2(r.vy,r.vx)*180/Math.PI).toFixed(0);
  var t = getAssignedTarget(r);
  var distToTarget = t ? dist(t,r).toFixed(0) : "-";
  var neighbors = state.robots.filter(function(o){ return o!==r &&
dist(o,r)<state.commRange; }).length;
  panel.innerHTML =
    '<div class="panel-head"><strong>ROBOT '+r.id+'</strong><button class="close-
btn" onclick="window.__closeRobotDetails()">✕</button></div>'+
    '<div class="detail-row"><span>Position</span><b>X='+Math.round(r.x)+',
Y='+Math.round(r.y)+'</b></div>'+
    '<div class="detail-row"><span>Velocity</span><b>'+speed+' m/s</b></div>'+
    '<div class="detail-row"><span>Direction</span><b>'+dir+'°</b></div>'+
    '<div class="detail-
row"><span>Battery</span><b>'+Math.round(r.battery)+'%</b></div>'+
    '<div class="detail-row"><span>Target</span><b>'+(t?t.id:"None")+'</b></div>'+
    '<div class="detail-row"><span>Dist. to Target</span><b>'+distToTarget+'
px</b></div>'+
    '<div class="detail-row"><span>Comm.
Neighbours</span><b>'+neighbors+'</b></div>'+
    '<div class="detail-row"><span>Status</span><b>'+r.status+'</b></div>';
}
window.__closeRobotDetails = function(){
  state.selectedRobotId = null;
  document.getElementById("robotDetailsPanel").classList.add("hidden");
};

/* ================= CHARTS ================= */
function drawChart(canvas, data, color){
  var ctx = canvas.getContext("2d");
  var w = canvas.width = canvas.clientWidth;
  var h = canvas.height = canvas.clientHeight;
  ctx.clearRect(0,0,w,h);
  ctx.strokeStyle = "rgba(255,255,255,0.06)";
  ctx.lineWidth = 1;
  for(var i=1;i<4;i++){ var y=h/4*i; ctx.beginPath(); ctx.moveTo(0,y); ctx.lineTo(w,y);
ctx.stroke(); }
  if(data.length<2) return;
  var maxVal = Math.max.apply(null, data.concat([1]))*1.15;
  ctx.beginPath();
```

```
  for(var idx=0; idx<data.length; idx++){
    var x = (idx/(data.length-1))*w;
    var yy = h - (data[idx]/maxVal)*h;
    if(idx===0) ctx.moveTo(x,yy); else ctx.lineTo(x,yy);
  }
  ctx.strokeStyle = color; ctx.lineWidth = 2; ctx.stroke();
  ctx.lineTo(w,h); ctx.lineTo(0,h); ctx.closePath();
  var grad = ctx.createLinearGradient(0,0,0,h);
  grad.addColorStop(0, color+"55");
  grad.addColorStop(1, color+"00");
  ctx.fillStyle = grad; ctx.fill();
  var last = data[data.length-1];
  ctx.fillStyle = color; ctx.font = "bold 11px Segoe UI";
  ctx.fillText(Math.round(last*10)/10, w-30, 13);
}
function sampleCharts(){
  state.charts.efficiency.push(computeEfficiency());
  state.charts.active.push(state.robots.filter(function(r){ return r.battery>0; }).length);
  state.charts.completed.push(state.metrics.completed);
  state.charts.collisions.push(state.metrics.collisions);
  ["efficiency","active","completed","collisions"].forEach(function(k){
    if(state.charts[k].length>40) state.charts[k].shift();
  });
  drawChart(document.getElementById("chartEfficiency"), state.charts.efficiency,
"#00e5ff");
  drawChart(document.getElementById("chartActive"), state.charts.active, "#22c55e");
  drawChart(document.getElementById("chartCompleted"), state.charts.completed,
"#ff9800");
  drawChart(document.getElementById("chartCollisions"), state.charts.collisions,
"#ff3b3b");
}

/* ================ RENDERING ================ */
var canvas = document.getElementById("simCanvas");
var ctx = canvas.getContext("2d");

function drawBackgroundGrid(){
  ctx.strokeStyle = "rgba(255,255,255,0.035)";
  ctx.lineWidth = 1;
  for(var x=0;x<WIDTH;x+=40){ ctx.beginPath(); ctx.moveTo(x,0);
ctx.lineTo(x,HEIGHT); ctx.stroke(); }
  for(var y=0;y<HEIGHT;y+=40){ ctx.beginPath(); ctx.moveTo(0,y);
ctx.lineTo(WIDTH,y); ctx.stroke(); }
}
function drawObstacles(){
```

```
  for(var i=0;i<state.obstacles.length;i++){
    var o = state.obstacles[i];
    var grad = ctx.createRadialGradient(o.x,o.y,0,o.x,o.y,o.r);
    grad.addColorStop(0,"rgba(255,90,90,0.35)");
    grad.addColorStop(1,"rgba(120,20,20,0.15)");
    ctx.beginPath(); ctx.arc(o.x,o.y,o.r+8,0,Math.PI*2); ctx.fillStyle=grad; ctx.fill();
    ctx.beginPath(); ctx.arc(o.x,o.y,o.r,0,Math.PI*2);
    ctx.fillStyle = "#3a1010"; ctx.fill();
    ctx.strokeStyle = "rgba(255,120,120,0.6)"; ctx.lineWidth=1.5; ctx.stroke();
  }
}
function drawCommLinks(){
  var now = performance.now();
  for(var i=0;i<state.commEdges.length;i++){
    var e = state.commEdges[i];
    var alpha = Math.max(0.08, 1-(e.d/state.commRange));
    ctx.beginPath();
    ctx.setLineDash([6,4]);
    ctx.lineDashOffset = -(now/50)%10;
    ctx.moveTo(e.a.x,e.a.y); ctx.lineTo(e.b.x,e.b.y);
    ctx.strokeStyle = "rgba(0,229,255,"+alpha.toFixed(2)+")";
    ctx.lineWidth = 1.4;
    ctx.stroke();
    ctx.setLineDash([]);
  }
}
function drawTargets(){
  var now = performance.now();
  var colors = {WAITING:"#5f7291", ASSIGNED:"#ff9800", "IN
PROGRESS":"#00e5ff", COMPLETED:"#22c55e"};
  for(var i=0;i<state.targets.length;i++){
    var t = state.targets[i];
    var pulse = 5+Math.sin(now/260+i)*3;
    var col = colors[t.status] || "#5f7291";
    ctx.beginPath(); ctx.arc(t.x,t.y,14+pulse,0,Math.PI*2);
    ctx.strokeStyle = col+"66"; ctx.lineWidth=2; ctx.stroke();
    ctx.beginPath(); ctx.arc(t.x,t.y,10,0,Math.PI*2);
    ctx.fillStyle = col+"33"; ctx.fill();
    ctx.strokeStyle = col; ctx.lineWidth=2; ctx.stroke();
    ctx.beginPath(); ctx.moveTo(t.x-6,t.y); ctx.lineTo(t.x+6,t.y);
    ctx.moveTo(t.x,t.y-6); ctx.lineTo(t.x,t.y+6);
    ctx.strokeStyle = col; ctx.lineWidth=1.5; ctx.stroke();
    ctx.fillStyle = "#dce8fb"; ctx.font="10px Segoe UI"; ctx.textAlign="center";
    ctx.fillText(t.id, t.x, t.y-22);
    if(t.status==="COMPLETED"){
```

```
      ctx.fillStyle="#22c55e"; ctx.font="bold 9px Segoe UI";
      ctx.fillText("✓ DONE", t.x, t.y+28);
    }
    ctx.textAlign="left";
  }
}
function drawRobots(){
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    var isLeader = (r.id===state.leaderId) && (state.algorithm==="leaderfollower" ||
state.formation==="leaderfollower");
    var isSelected = r.id===state.selectedRobotId;
    var color = "#00e5ff";
    if(r.status==="LOW BATTERY") color="#ff9800";
    if(r.status==="OFFLINE") color="#5f7291";
    if(r.collisionFlash>0.4) color="#ff3b3b";

    if(isSelected){
      ctx.beginPath();
ctx.arc(r.x,r.y,18+Math.sin(performance.now()/150)*2,0,Math.PI*2);
      ctx.strokeStyle="rgba(0,229,255,0.8)"; ctx.lineWidth=2; ctx.stroke();
    }
    if(isLeader){
      ctx.beginPath(); ctx.arc(r.x,r.y,15,0,Math.PI*2);
      ctx.strokeStyle="#ffd54a"; ctx.lineWidth=2.2; ctx.stroke();
    }

    var ang = Math.atan2(r.vy,r.vx);
    ctx.beginPath(); ctx.arc(r.x,r.y,9,0,Math.PI*2);
    var g = ctx.createRadialGradient(r.x-2,r.y-2,1,r.x,r.y,10);
    g.addColorStop(0,"#ffffff"); g.addColorStop(0.25,color);
g.addColorStop(1,"#0a1428");
    ctx.fillStyle=g; ctx.fill();
    ctx.strokeStyle=color; ctx.lineWidth=1.5; ctx.stroke();

    ctx.beginPath();
    ctx.moveTo(r.x+Math.cos(ang)*9, r.y+Math.sin(ang)*9);
    ctx.lineTo(r.x+Math.cos(ang)*15, r.y+Math.sin(ang)*15);
    ctx.strokeStyle = color; ctx.lineWidth=2; ctx.stroke();

    ctx.fillStyle="#cfe8ff"; ctx.font="9px Segoe UI"; ctx.textAlign="center";
    ctx.fillText(r.id, r.x, r.y-14);

    var battWidth = 16;
    ctx.fillStyle="rgba(0,0,0,0.5)"; ctx.fillRect(r.x-battWidth/2, r.y+12, battWidth, 3);
```

```
    var bcol = r.battery<20 ? "#ff3b3b" : (r.battery<50 ? "#ff9800" : "#22c55e");
    ctx.fillStyle=bcol; ctx.fillRect(r.x-battWidth/2, r.y+12, battWidth*(r.battery/100),
3);
    ctx.textAlign="left";
  }
}
function render(){
  ctx.clearRect(0,0,WIDTH,HEIGHT);
  drawBackgroundGrid();
  drawObstacles();
  drawCommLinks();
  drawTargets();
  drawRobots();
}

/* ================ CANVAS INTERACTION ================ */
function getCanvasCoords(evt){
  var rect = canvas.getBoundingClientRect();
  var scaleX = WIDTH/rect.width, scaleY = HEIGHT/rect.height;
  return { x:(evt.clientX-rect.left)*scaleX, y:(evt.clientY-rect.top)*scaleY };
}
function addObstacle(x,y){
  state.obstacles.push({x:x, y:y, r:14+Math.random()*14});
  logEvent("Obstacle added at ("+Math.round(x)+", "+Math.round(y)+")");
}
function addTarget(x,y){
  var t = {id:"T"+(state.targetCounter++), x:x, y:y, status:"WAITING"};
  state.targets.push(t);
  logEvent("New target "+t.id+" created by user");
  assignTargets();
}
function removeTargetAt(x,y){
  for(var i=0;i<state.targets.length;i++){
    var t = state.targets[i];
    if(Math.hypot(t.x-x,t.y-y)<20){
      state.targets.splice(i,1);
      logEvent("Target "+t.id+" removed by user");
      return true;
    }
  }
  return false;
}

var lastClickTime=0, lastClickPos=null;
canvas.addEventListener("click", function(e){
```

```
  var now = Date.now();
  var p = getCanvasCoords(e);
  if(now-lastClickTime<350 && lastClickPos && Math.hypot(p.x-lastClickPos.x,p.y-
lastClickPos.y)<20){
    removeTargetAt(p.x,p.y);
    lastClickTime=0;
    render();
    return;
  }
  lastClickTime = now; lastClickPos = p;

  var hit=null;
  for(var i=0;i<state.robots.length;i++){
    if(Math.hypot(state.robots[i].x-p.x, state.robots[i].y-p.y)<16){ hit=state.robots[i];
break; }
  }
  if(hit){
    state.selectedRobotId = hit.id;
    document.getElementById("robotDetailsPanel").classList.remove("hidden");
    updateRobotDetailsPanel();
    render();
    return;
  }
  if(e.shiftKey){ addTarget(p.x,p.y); } else { addObstacle(p.x,p.y); }
  render();
});

/* ================= CSV EXPORT ================= */
function exportCSV(){
  var rows = [["Time(s)","RobotID","X","Y","Velocity","Target","Battery","Status"]];
  var t = state.startTime ? ((performance.now()-state.startTime)/1000).toFixed(1) :
"0.0";
  for(var i=0;i<state.robots.length;i++){
    var r = state.robots[i];
    var speed = Math.hypot(r.vx,r.vy).toFixed(2);
    var tg = getAssignedTarget(r);
    rows.push([t, r.id, Math.round(r.x), Math.round(r.y), speed, tg?tg.id:"-",
Math.round(r.battery), r.status]);
  }
  var csv = rows.map(function(row){ return row.join(","); }).join("\n");
  var blob = new Blob([csv], {type:"text/csv"});
  var url = URL.createObjectURL(blob);
  var a = document.createElement("a");
  a.href=url; a.download="swarm_simulation_data.csv";
  document.body.appendChild(a); a.click(); document.body.removeChild(a);
```

```
  URL.revokeObjectURL(url);
  logEvent("Simulation data exported to CSV");
}

/* ================ CONTROLS / BUTTONS ================ */
function updateSystemStatus(){
  var el = document.getElementById("systemStatus");
  el.className = "status-badge";
  if(state.emergency){ el.classList.add("emergency"); el.textContent="
EMERGENCY STOP ACTIVATED"; }
  else if(state.running){ el.textContent="  SYSTEM ONLINE"; }
  else { el.classList.add("paused"); el.textContent="  SYSTEM PAUSED"; }
}
function startSim(){
  state.emergency=false;
  document.getElementById("emergencyBanner").classList.remove("show");
  state.running=true;
  if(!state.startTime) state.startTime=performance.now();
  logEvent("Swarm deployed — simulation started");
  updateSystemStatus();
}
function pauseSim(){
  state.running=false;
  logEvent("Simulation paused");
  updateSystemStatus();
}
function resetSim(){
  state.running=false;
  state.emergency=false;
  document.getElementById("emergencyBanner").classList.remove("show");
  initSimulation();
  updateSystemStatus();
}
function randomizeSim(){
  initSimulation();
  logEvent("Swarm configuration randomized");
}
function emergencyStopSim(){
  state.running=false;
  state.emergency=true;
  for(var i=0;i<state.robots.length;i++){ state.robots[i].vx=0; state.robots[i].vy=0; }
  document.getElementById("emergencyBanner").classList.add("show");
  logEvent("EMERGENCY STOP ACTIVATED — all robots halted");
  updateSystemStatus();
}
```

```
document.getElementById("btnStart").addEventListener("click", startSim);
document.getElementById("btnPause").addEventListener("click", pauseSim);
document.getElementById("btnReset").addEventListener("click", resetSim);
document.getElementById("btnRandom").addEventListener("click", randomizeSim);
document.getElementById("btnEmergency").addEventListener("click",
emergencyStopSim);
document.getElementById("btnExport").addEventListener("click", exportCSV);

/* Sliders */
var sliderRobots = document.getElementById("sliderRobots"), valRobots =
document.getElementById("valRobots");
sliderRobots.addEventListener("input", function(){
valRobots.textContent=sliderRobots.value; });
sliderRobots.addEventListener("change", function(){
state.config.numRobots=parseInt(sliderRobots.value,10);
adjustRobotCount(state.config.numRobots); });

var sliderTargets = document.getElementById("sliderTargets"), valTargets =
document.getElementById("valTargets");
sliderTargets.addEventListener("input", function(){
valTargets.textContent=sliderTargets.value; });
sliderTargets.addEventListener("change", function(){
state.config.numTargets=parseInt(sliderTargets.value,10);
adjustTargetCount(state.config.numTargets); });

var speedMap=[0.5,1,2,3];
var sliderSpeed = document.getElementById("sliderSpeed"), valSpeed =
document.getElementById("valSpeed");
sliderSpeed.addEventListener("input", function(){
  state.speed = speedMap[parseInt(sliderSpeed.value,10)];
  valSpeed.textContent = state.speed+"x";
});

var sliderComm = document.getElementById("sliderComm"), valComm =
document.getElementById("valComm");
sliderComm.addEventListener("input", function(){
  state.commRange = parseInt(sliderComm.value,10);
  valComm.textContent = state.commRange+" px";
});

var obstacleMap=["low","medium","high"],
obstacleLabels=["Low","Medium","High"];
var sliderObstacle = document.getElementById("sliderObstacle"), valObstacle =
document.getElementById("valObstacle");
```

```
sliderObstacle.addEventListener("input", function(){ valObstacle.textContent =
obstacleLabels[parseInt(sliderObstacle.value,10)]; });
sliderObstacle.addEventListener("change", function(){
  state.config.obstacleDensity = obstacleMap[parseInt(sliderObstacle.value,10)];
  regenerateObstacles();
  logEvent("Obstacle density set to
"+obstacleLabels[parseInt(sliderObstacle.value,10)].toUpperCase()+"
("+state.obstacles.length+" obstacles)");
  render();
});

var selectAlgorithm = document.getElementById("selectAlgorithm");
selectAlgorithm.addEventListener("change", function(){
  state.algorithm = selectAlgorithm.value;
  var d = ALGO_DESC[state.algorithm];
  document.getElementById("algoDesc").innerHTML =
"<b>"+d.name+"</b>"+d.text;
  logEvent("Coordination algorithm changed to "+d.name);
  assignTargets();
});

var selectFormation = document.getElementById("selectFormation");
var formationLabels={free:"Free Swarm", line:"Line Formation", v:"V Formation",
circle:"Circle Formation", grid:"Grid Formation", leaderfollower:"Leader-Follower"};
selectFormation.addEventListener("change", function(){
  state.formation = selectFormation.value;
  logEvent("Formation mode set to "+formationLabels[state.formation]);
});

function bindWeightSlider(id, key, labelId){
  var el = document.getElementById(id), lab = document.getElementById(labelId);
  el.addEventListener("input", function(){
    state.weights[key] = parseFloat(el.value);
    lab.textContent = el.value;
  });
}
bindWeightSlider("wSep","sep","valWSep");
bindWeightSlider("wAlign","align","valWAlign");
bindWeightSlider("wCoh","coh","valWCoh");
bindWeightSlider("wTarget","target","valWTarget");
bindWeightSlider("wObs","obstacle","valWObs");

/* ================ MAIN LOOP ================ */
var lastTime = performance.now();
var frameCount = 0;
```

```
function animate(now){
  requestAnimationFrame(animate);
  var rawDt = Math.min((now-lastTime)/16.6667, 3);
  lastTime = now;
  if(state.running && !state.emergency){
    var dt = rawDt*state.speed;
    updateSimulation(dt);
  }
  frameCount++;
  if(frameCount % 15 === 0){
    assignTargets();
    refreshPanels();
  }
  if(frameCount % 30 === 0){
    sampleCharts();
  }
  render();
}

/* ================ BOOTSTRAP ================ */
initSimulation();
updateSystemStatus();
requestAnimationFrame(animate);

})();
</script>
</body>
</html>
```